OnBuch+

SIGNED COURSE • PREMIUM

ICT • UPPER SIXTH | UPPER SIXTH — ICT

CHAPTER 15

DEVELOP SIMPLE WEB PAGES

SUGGESTED TIME: 2 PERIODS

IN THIS CHAPTER

This chapter introduces the World Wide Web and teaches learners how to develop simple web pages using HTML, the standard markup language of the Web. Through guided practice, learners will structure documents, format text, insert links, images, lists and tables, and finally integrate all these skills to build a complete mini-website for a real-life Cameroonian context.

- By the end of this chapter I can explain what the World Wide Web, a website, a web page, a browser and HTML are.
- By the end of this chapter I can describe the structure of an HTML document (doctype, html, head, body) and the role of tags and attributes.
- By the end of this chapter I can create and save a simple HTML file with a text editor and view it in a web browser.
- By the end of this chapter I can use basic text-formatting tags: headings, paragraphs, line breaks, bold, italic and horizontal rules.
- By the end of this chapter I can insert hyperlinks, images, ordered and unordered lists into a web page.
- By the end of this chapter I can build a table with rows, columns, headers and captions in HTML.

DOWNLOAD ONBUCH
Free • Google Play
AI tutor, past papers, results

GO ONBUCH+
All signed courses, unlimited AI tutor, PDF sheets — OnBuch+ tab in the app

THIS SIGNED COURSE CONTAINS

COURSE complete, illustrated, step by step

METHODS and worked examples

EXERCISES exam style + solutions

SUMMARY SHEET the essentials to remember

CONTENTS

1. The Web and the structure of an HTML document
2. Formatting text and structuring content
3. Links, images, lists and tables
4. Forms, validation and integration project
5. Integration activity
6. Methods and worked examples
7. Exercises
8. Solutions to the exercises
9. Summary sheet

CHAPTER OBJECTIVES

- By the end of this chapter I can explain what the World Wide Web, a website, a web page, a browser and HTML are.
- By the end of this chapter I can describe the structure of an HTML document (doctype, html, head, body) and the role of tags and attributes.
- By the end of this chapter I can create and save a simple HTML file with a text editor and view it in a web browser.
- By the end of this chapter I can use basic text-formatting tags: headings, paragraphs, line breaks, bold, italic and horizontal rules.
- By the end of this chapter I can insert hyperlinks, images, ordered and unordered lists into a web page.
- By the end of this chapter I can build a table with rows, columns, headers and captions in HTML.
- By the end of this chapter I can create a simple form with common input elements (text field, radio button, checkbox, submit button).
- By the end of this chapter I can combine several HTML elements to develop a small multi-page website on a familiar topic.
- By the end of this chapter I can check and correct common HTML errors (unclosed tags, wrong nesting, missing attributes).

PREREQUISITES

- Basic file management: creating folders, saving and renaming files, using file extensions (Lower Sixth, ICT on computer environment).
- Use of a text editor such as Notepad and of a web browser (Lower Sixth).
- Notion of the Internet, websites and URLs seen in the Lower Sixth chapter on networks and the Internet.
- Basic keyboard skills and ability to type special characters (<, >, /, ”).

1 THE WEB AND THE STRUCTURE OF AN HTML DOCUMENT

During the last general assembly of your school, the principal announced that the school needs a website to publish results, announcements and the time-table, but the web agency in Yaoundé quoted 350 000 FCFA for the job. A classmate whispers: "But a simple web page is just a text file with special marks — we could do it ourselves!" Is that really true? What exactly does a browser like Chrome or Firefox read when it displays a page? In this chapter you will discover that behind every page you visit on your phone at the cybercafé lies a simple, readable language called **HTML** — and you will learn to write it yourself.

1.1 Key Web concepts

Before writing any code, we must clarify the vocabulary used every day in the cybercafés of Douala, Yaoundé, Buea or Bamenda. Many students confuse the **Internet** and the **World Wide Web (WWW)**.

DEFINITION **Internet**

The **Internet** is the global hardware infrastructure: a network of networks connecting millions of computers, routers, fibre-optic cables (like the SAT-3/WASC cable landing in Limbe), satellites and mobile towers. It transports data packets using the **IP** (Internet Protocol).

DEFINITION **World Wide Web (WWW)**

The **World Wide Web** (or simply **the Web**) is an information system that runs on top of the Internet. It consists of **web pages** linked by hyperlinks, accessed via **HTTP/HTTPS** protocols. The Web was invented by Tim Berners-Lee at CERN in 1989.

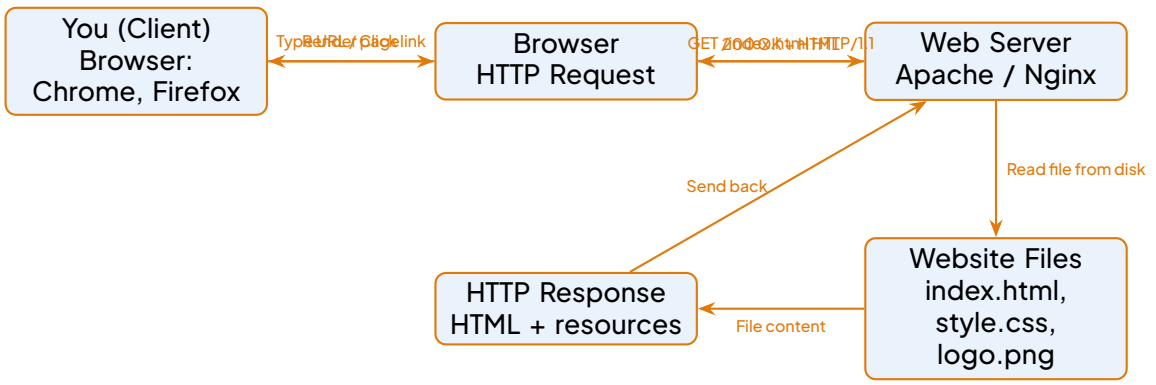
DEFINITION **Web browser**

A **web browser** (Chrome, Firefox, Edge, Safari, Opera Mini) is a client software that sends **HTTP requests** to a server, receives **HTML documents** (plus CSS, JavaScript, images), interprets them and renders a visual page on your screen.

DEFINITION **Web server**

A **web server** is a computer (or software like Apache, Nginx, IIS) permanently connected

to the Internet, storing web site files. It listens for HTTP requests and responds with the requested resources (HTML files, images, etc.).



Client–server model of the Web: the browser requests a resource, the server delivers the HTML file, the browser renders it.

DEFINITION **Website, Web page, Home page**

- A **website** is a collection of related web pages under a common domain name (e.g. `www.gov.cm`, `www.mtn.cm`).
- A **web page** is a single document (usually an `.html` file) identified by a unique **URL**.
- The **home page** (often `index.html`) is the entry page of a website.

DEFINITION **URL (Uniform Resource Locator)**

A **URL** is the address of a resource on the Web. Example: `https://www.gceboard.cm/results/2024.pdf`

Part	Example
Scheme (protocol)	`https://`
Domain name	`www.gceboard.cm`
Path	`/results/2024.pdf`

1.2 What is HTML?

DEFINITION **HTML (HyperText Markup Language)**

HTML is a **markup language** (not a programming language) used to structure and give meaning to the content of a web page. It uses **tags** to mark up text, images, links, etc. The current standard is HTML5 (W3C Recommendation, 2014; WHATWG Living Standard).

KEY POINTS **HTML vs Programming Languages**

- HTML has no variables, no loops, no functions, no logic. It only describes content.
- Browsers interpret HTML; they do not compile it.
- To add interactivity (calculations, form validation, animations) we later combine HTML with CSS (styling) and JavaScript (programming).

DEFINITION **Tag, Element, Attribute**

- A **tag** is a keyword enclosed in angle brackets: `<p>` (opening tag), `</p>` (closing tag).
- An **element** = opening tag + content + closing tag. Example: `<p>Welcome to GHS Buea</p>`.
- An **attribute** gives extra information about an element, written in the opening tag as `name="value"`. Example: `<img src="logo.png" alt="School logo">`.

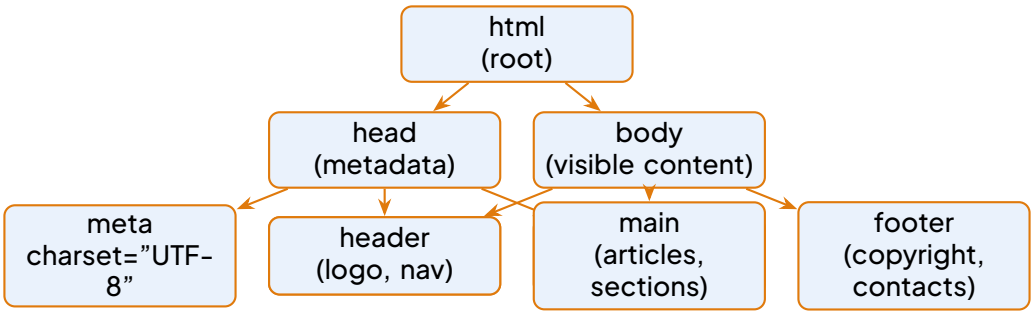
PROPERTY **Syntax rules for HTML tags**

1. Tags are case-insensitive (`<P>` = `<p>`), but lowercase is the standard (HTML5).
2. Most elements have an opening and a closing tag. Void elements (self-closing) have no content and no closing tag: `<br>`, `<hr>`, `<img>`, `<meta>`, `<input>`. In HTML5 the trailing slash is optional: `<br>` or `<br />`.
3. Attribute values should be quoted (double quotes preferred): `class="highlight"`.
4. Tags must be properly nested: `<p><strong>Text</strong></p>` (correct), `<p><strong>Text</p></strong>` (incorrect).

1.3 Anatomy of an HTML document

Every valid HTML5 document follows a strict skeleton. Think of it as the architectural plan of a

house: the <html> element is the whole plot, <head> holds the legal papers (metadata), <body> is the living space where visitors see things.



Tree diagram of the HTML document skeleton.

EXEMPLE **Minimal HTML5 document**

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>GHS Buea - Home</title>
</head>
<body>
  <h1>Welcome to Government High School Buea</h1>
  <p>Excellence, Discipline, Integrity.</p>
</body>
</html>
```

DEFINITION **The DOCTYPE declaration**

<!DOCTYPE html> must be the very first line. It tells the browser: "This document follows the HTML5 standard; render in standards mode." Without it, browsers may switch to quirks mode (old, buggy rendering).

DEFINITION **The <html> root element**

All other elements are descendants of <html>. The lang attribute (e.g. lang="en" or lang="fr") helps screen readers and search engines.

DEFINITION **The `<head>` — metadata container**

The head holds metadata (data about the page) — not displayed directly but crucial:

- `<title>`: shown in the browser tab, bookmarks, search results. Mandatory.
- `<meta charset="UTF-8">`: declares the character encoding. Without it, accents (`é, è, à, ô`) and special characters (CFA symbol) may appear as garbage (☰).
- `<meta name="viewport" content="width=device-width, initial-scale=1.0">`: essential for responsive design on mobile phones.
- `<link rel="stylesheet" href="style.css">`: links an external CSS file.
- `<script src="app.js" defer></script>`: links JavaScript (defer ensures HTML is parsed first).

WARNING **Forgetting `<meta charset="UTF-8">`**

If you write French or Cameroonian language text (Bamileke, Fulfulde, Duala) without this tag, the browser may guess a wrong encoding (e.g. ISO-8859-1) and display instead of `é, ë, ŋ,  `. Always include it as the first child of `<head>`.

DEFINITION **The `<body>` — visible content**

Everything the user sees (headings, paragraphs, images, links, tables, forms, videos) goes inside `<body>`. There is exactly one `<body>` per document.

1.4 Creating and viewing your first HTML page

You do not need expensive software. A plain text editor and a browser are enough.

METHOD **5 steps to create and view your first HTML page**

1. Open a text editor: Notepad (Windows), TextEdit (Mac, in Plain Text mode), or better: Notepad++ (free, syntax highlighting) or VS Code.
2. Write the HTML skeleton (copy the minimal example above).
3. Save the file with the .html extension: e.g. `index.html`. Not `index.html.txt`, `index.htm`, `index.docx`.
4. Open the file in a browser: double-click it, or drag it into Chrome/Firefox, or right-

click → Open with → Browser.

5. Edit → Save → Refresh (F5): every change requires saving the file and refreshing the browser.

WARNING **File extension trap**

If you save as `page.html.txt` (Windows hides known extensions by default), the browser will open it as plain text, showing your tags instead of rendering the page. Always verify the extension (enable File name extensions in Windows Explorer View tab).

WARNING **Missing closing tag — the #1 beginner error**

Writing `<p>First paragraph` without `</p>` may seem to work because browsers are forgiving, but it breaks nesting and makes later CSS/JavaScript unpredictable. Always close every non-void element.

DEFINITION **HTML Comments**

Comments are ignored by the browser but help developers. Syntax: `<!-- This is a comment -->`. They can span multiple lines. Never put sensitive data (passwords, API keys) in comments — they are visible in View Source.

KEY POINTS **Good indentation habits**

Indent child elements by 2 or 4 spaces (or a tab). It makes the tree structure visible and debugging easy.

```
<body>
  <header>
    <h1>Title</h1>
  </header>
  <main>
    <p>Content</p>
  </main>
</body>
```

1.5 Worked example: A simple school notice page

WORKED EXAMPLE 1 **Create a page displaying the GCE Advanced Level time-table notice**

Statement: Write the complete HTML5 code for a page titled `GCE A-Level Time-Table 2025` that shows:

- A main heading: `Government Bilingual High School Yaoundé`
- A sub-heading: `Advanced Level Time-Table --- June 2025 Session`
- A paragraph: `All candidates must arrive 30 minutes before each paper.`
- A horizontal rule (`<hr>`) separating the notice from a footer line: `Published by the Divisional Delegation of Secondary Education, Mfoundi.`

Requirements: Proper DOCTYPE, charset, language, indentation, and a comment describing the page.

SOLUTION

Detailed solution:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>GCE A-Level Time-Table 2025</title>
  <!-- School notice page for the June 2025 session -->
</head>
<body>
  <header>
    <h1>Government Bilingual High School Yaoundé</h1>
  </header>
  <main>
    <h2>Advanced Level Time-Table --- June 2025 Session</h2>
    <p>All candidates must arrive 30 minutes before each paper.</p>
    <hr>
    <footer>
      <p>Published by the Divisional Delegation of Secondary Education, Mfoundi.</p>
    </footer>
  </main>
</body>
</html>
```

Explanation:

- Line 1: DOCTYPE for HTML5.
- Line 2: Root element with English language.
- Lines 3–6: Head with UTF-8 charset (critical for `é` in `Yaoundé` and `Mfoundi`), title, and a comment.
- Lines 8–18: Body with semantic elements (`header`, `main`, `footer`) for accessibility and SEO.

- `<hr>` is a void element (thematic break).

1.6 Summary of key points

KEY POINTS **What you must remember from this section**

- The Web is an information system on the Internet; browsers (clients) request HTML from servers via HTTP.
- HTML is a markup language (tags, elements, attributes) — not a programming language.
- Every HTML5 document starts with `<!DOCTYPE html>` and contains `<html>`, `<head>` (metadata, title, charset), `<body>` (visible content).
- `<meta charset="UTF-8">` is mandatory for correct display of accents and special characters.
- Workflow: Editor → Save as `.html` → Open in browser → Refresh.
- Always close tags, nest properly, indent, and use comments for maintenance.

DID YOU KNOW? **Cameroon's Web history**

Cameroon's first connection to the Internet was established in 1996 via the ANTIC (Agence Nationale des Technologies de l'Information et de la Communication). The `.cm` domain is managed by ANTIC. Today, mobile operators (MTN, Orange, Nexttel) provide 3G/4G access in most cities, making the Web accessible even in rural cybercafés. Many government services (e.g. `www.minfopra.gov.cm` for civil service exams) now publish results online — built with the same HTML you are learning.

EXERCISE 1 **Practice: Your first page**

Create a file `myfirst.html` that displays:

1. Title in browser tab: `My ICT Portfolio`
2. Main heading: `Upper Sixth ICT --- Web Development`
3. Paragraph: `I am learning HTML at` *`Government High School`* `[your school name].`
4. A horizontal rule.

5. **Footer line:** `© 2025 [Your Name] --- All rights reserved.`

Open it in your browser, take a screenshot, and show it to your teacher.

SOLUTION **Exercise 1**

The solution follows the exact skeleton taught. Key checks:

- File saved as `myfirst.html` (not .txt).
- `<!DOCTYPE html>` on line 1.
- `<meta charset="UTF-8">` present.
- Proper nesting: `html > head, body > header/main/footer > h1/p/hr`.
- No missing closing tags.

2 FORMATTING TEXT AND STRUCTURING CONTENT

In the previous section we discovered how the Web works and we built the skeleton of a valid HTML document: the **doctype**, the **head** and the **body**. Our first page, however, was just one line of text. A real school website, like the page of a bilingual high school in Yaoundé, needs titles, paragraphs, quotations and well-structured announcements. In this section we learn the HTML tags that give structure and meaning to text, and we see exactly how the browser interprets what we type.

2.1 Headings: `<h1>` to `<h6>`

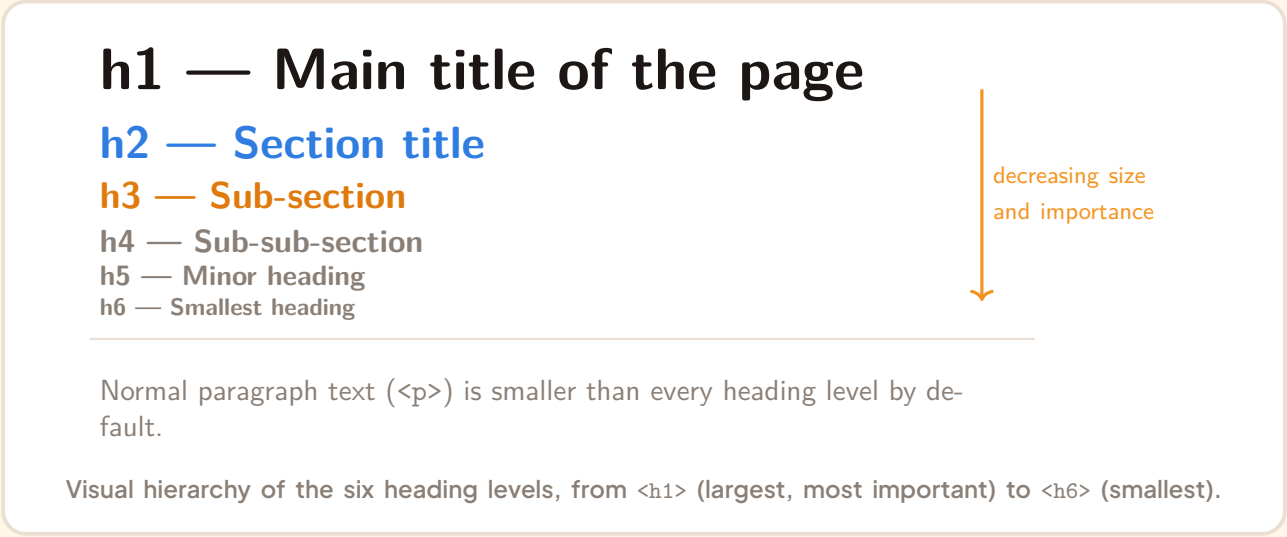
Imagine the notice board of your school. The biggest title announces "End-of-year results", then smaller subtitles announce "Form Five", "Lower Sixth", and so on. Readers understand the organisation of the information at a glance, simply because of the size and order of the titles. HTML offers exactly the same mechanism with six levels of headings.

PROPERTY **The six heading levels**

HTML provides six heading tags, `<h1>`, `<h2>`, `<h3>`, `<h4>`, `<h5>` and `<h6>`, which define six levels of importance. `<h1>` is the most important (rendered largest by the browser) and `<h6>` the least important (rendered smallest). Headings are **block-level** elements: each one starts on a new line, with vertical space around it.

Two rules of good practice follow from this property. First, a page should contain **a single** `<h1>`, which states the main subject of the page, just as an essay has one title. Second, the levels

should be used in order: an <h2> subdivides the <h1>, an <h3> subdivides an <h2>, and one should not jump from <h1> directly to <h4> simply to obtain a smaller font. The size of a heading is a matter of style, not of structure; at GCE level, headings must reflect the logical outline of the document.



Visual hierarchy of the six heading levels, from <h1> (largest, most important) to <h6> (smallest).

2.2 Paragraphs, line breaks and horizontal rules

Text in HTML is organised into **paragraphs** using the <p> tag. The browser automatically inserts vertical space before and after each paragraph, which separates ideas visually.

Sometimes we want to force a new line inside a paragraph, for example in a postal address or a poem. This is the role of the
 tag (break). The <hr> tag (horizontal rule) draws a horizontal line across the page, useful to separate two distinct parts of a document, such as two announcements.

WARNING **Empty tags:
 and <hr>**

 and <hr> are **empty tags**: they have no closing tag and no content. Writing
</br> or <hr></hr> is incorrect. In the HTML used at GCE level, simply write
 and <hr> alone. (In the stricter XHTML style you may meet the self-closing forms
 and <hr/>; both are accepted by modern browsers.)

WARNING **Browsers collapse spaces and line breaks**

A very common beginner's mistake is to press Enter several times in the code, or to type many spaces, hoping to space out the text. The browser collapses (merges) any sequence of spaces, tabs and line breaks typed in the source code into a single space. To obtain a new paragraph, use <p>; to force a line break, use
. Never try to format a page with the space bar.

WARNING **Tags must nest properly**

When several tags apply to the same text, they must be nested like brackets in mathematics: the tag opened last is closed first. Write `<p><strong>text</strong></p>`, never `<p><strong>text</p></strong>`. Overlapping tags are a classic source of errors detected by validators and penalised in practical examinations.

2.3 Inline formatting and semantic markup

Inside a paragraph we can emphasise words with inline tags, which do not start a new line:

Tag	Usual rendering	Meaning / use
`<b>`	**bold**	Visually bold text, no special importance
`<strong>`	**bold**	Strong importance (semantic)
`<i>`	italic	Visually italic text
`<em>`	italic	Emphasis (semantic)
`<u>`	underlined	Underlined text (avoid: looks like a link)
`<sup>`	superscript	Exponents, ordinals: x^2, 1st
`<sub>`	subscript	Chemical formulae: H_2O
`<mark>`	highlighted	Text highlighted like a marker pen

Why do two pairs of tags, `<b>`/`<strong>` and `<i>`/`<em>`, produce the same visual result? The difference is not on the screen but in the meaning carried by the code.

DEFINITION **Semantic markup**

Semantic markup consists in choosing tags according to the meaning of the content rather than its visual appearance. `<strong>` says "this text is important", whereas `<b>` says only "draw this text in bold". Similarly, `<em>` expresses emphasis while `<i>` expresses mere italics.

DID YOU KNOW? **Why semantic markup matters**

Semantic tags are not a purist's luxury. Screen readers used by visually impaired people pronounce `<strong>` text with a stronger intonation, search engines give more weight to words inside `<strong>` and headings, and a style sheet can later restyle all important words at once. For the GCE, remember: prefer `<strong>` to `<b>` and `<em>` to `<i>` whenever the formatting expresses importance or emphasis, and reserve `<b>`/`<i>` for purely visual effects.

2.4 Quotations and preformatted text

To quote a long passage, for example an extract from a speech or a regulation, HTML provides the block-level tag `<blockquote>`. Browsers usually render it as an indented block, clearly separated from the surrounding paragraphs. For a short quotation inside a sentence, the inline tag `<q>` exists, which adds quotation marks automatically.

The `<pre>` tag (preformatted) is special: it is the **only** common element in which the browser preserves the spaces and line breaks exactly as typed, displayed in a monospace font. It is therefore ideal for displaying computer code, a poem with a precise layout, or a small diagram made of characters.

EXEMPLE **Quotation and preformatted text**

HTML code	Effect
`<blockquote>Education is the most powerful weapon.</blockquote>`	Indented quotation block
`<pre>H2O + CO2= life</pre>`	Text displayed exactly as typed, line breaks kept
`<p>As the motto says, <q>Discipline leads to success</q>.</p>`	Short inline quotation with automatic quotation marks

2.5 Special characters and entities

Suppose you want to display the sentence “The tag `<p>` creates a paragraph” on your page. If you type the characters < and > directly, the browser believes a tag is starting and your text disappears or is mangled. The characters <, > and `&` are reserved by the HTML language itself. The solution is to write them as **entities**: short codes that begin with `&` and end with a semicolon.

METHOD **Displaying a reserved character with an entity**

1. Identify the reserved character you want to show: <, > or `&`.
2. Replace it by its entity: `<` for < (less than), `>` for > (greater than), `&` for `&` (ampersand).

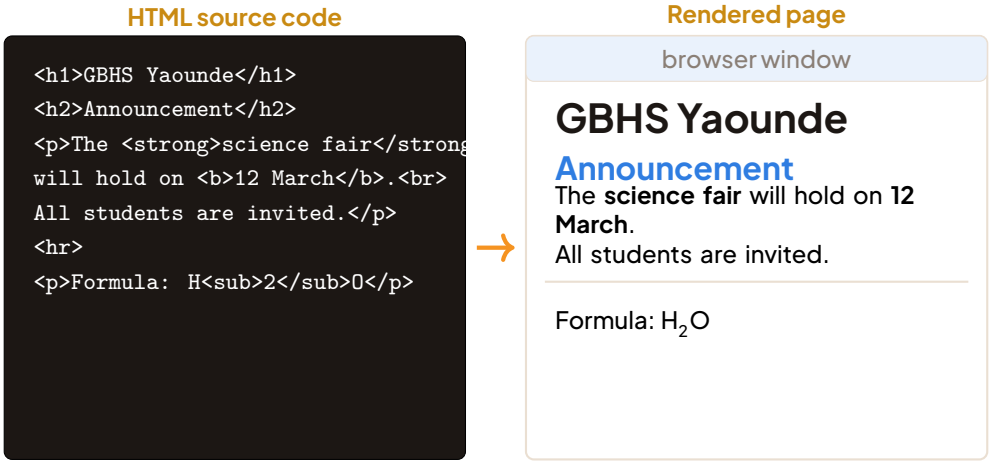
3. Check that every entity ends with a semicolon; without it, the browser may not recognise the entity.
4. For a non-breaking space (a space that prevents a line break, e.g. between a number and its unit), use ` `.

Accented letters needed in French (é, è, à, ç…) can be typed directly if the file is saved in **UTF-8** encoding and the page declares `<meta charset="UTF-8">` in its head, as we saw in the previous section. Entities also exist for them (`é` gives é, `à` gives à), which remains a safe alternative when the encoding is uncertain.

Entity	Displays	Remark
`<`	<	Less-than sign, starts a tag
`>`	>	Greater-than sign, ends a tag
`&`	&	Ampersand, starts an entity
` `	(space)	Non-breaking space
`é`	é	e with acute accent

2.6 From code to screen: a worked example

The figure below summarises the essential idea of this section: the browser reads the tags in the source code (left) and produces a formatted rendering (right).



Left: HTML source code using headings, paragraph, `<strong>`, `<br>`, `<hr>` and `<sub>`. Right: the corresponding browser rendering.

WORKED EXAMPLE 2 **Marking up a school announcement**

The principal of a high school in Bamenda asks you to publish the following announcement on the school website: a main title “GBHS Bamenda — Notice Board”, a sub-title “General Assembly”, a paragraph announcing that the general assembly of parents will take place on Friday 14 November (the date must appear in bold because it is essential), a horizontal line, then a short paragraph “Attendance is compulsory for all parents.” Write the HTML code of the body.

SOLUTION

We apply the rules of the section: one single `<h1>` for the page title, an `<h2>` for the sub-title, paragraphs in `<p>`, the important date in `<strong>` (semantic importance, not just visual bold), the empty tag `<hr>` to separate, and `<em>` for the emphasis on “compulsory”.

HTML code of the body

```
<h1>GBHS Bamenda --- Notice Board</h1>
<h2>General Assembly</h2>
<p>The general assembly of parents will take place on <strong>Friday 14 November</strong> in the school hall.</p>
<hr>
<p>Attendance is <em>compulsory</em> for all parents.</p>
```

Checks: the `<hr>` has no closing tag; `<strong>` is preferred to `<b>` because the date is important; there is exactly one `<h1>`; every opened tag is closed in the correct order.

EXERCISE 2 **Your turn: the canteen menu**

Write the HTML code of a page body for the school canteen containing: one main title “College Canteen”, a sub-title “Menu of the week”, a paragraph with the sentence “Meals are served from 11th hour” (with “th” in superscript using `<sup>`), a line break inside the paragraph to add “Price: 500 FCFA” on the next line, a horizontal rule, and the chemical formula of water H_2O in a final paragraph.

SOLUTION **Exercise 1**

```
<h1>College Canteen</h1>
<h2>Menu of the week</h2>
<p>Meals are served from 11<sup>th</sup> hour.<br>Price: 500 FCFA.</p>
<hr>
<p>Water formula: H<sub>2</sub>O</p>
```

Note the single `<h1>`, the empty tags `<br>` and `<hr>` without closing tags, and the correct

use of <sup> and <sub>.

KEY POINTS **Key points of the section**

- Six heading levels <h1> to <h6>; one single <h1> per page; levels used in logical order.
- <p> for paragraphs,
 for a forced line break, <hr> for a horizontal rule;
 and <hr> are empty tags.
- Prefer and (semantic) to <b> and <i> (presentational); <sup>, <sub>, <mark> complete the inline toolbox.
- <blockquote> for long quotations, <q> for short inline quotations, <pre> to preserve spaces and line breaks exactly.
- Reserved characters are written as entities: <, >, &, .
- The browser collapses spaces and line breaks typed in the code: structure comes from tags, never from the space bar.
- Tags must nest properly: the tag opened last is closed first.

Now that our pages have well-structured and correctly formatted text, the next step is to make them richer and more interactive: in the following section we will insert **hyperlinks**, **images**, **lists** and **tables**, the four ingredients that transform a plain document into a true web page.

3 LINKS, IMAGES, LISTS AND TABLES

In the previous section you learnt how to format text and structure content with headings, paragraphs and emphasis. A page made only of text, however rich, remains an isolated island. The real power of the Web comes from three things: pages that **link** to one another, pages enriched with **images**, and content organised into **lists** and **tables**. In this section we add these four building blocks to your toolkit, and we finish with a mini-project: a page presenting your region.

3.1 Hyperlinks: connecting pages together

Intuition. When you browse the website of the University of Yaoundé I or a news site like crtv.cm, you constantly click on underlined text or buttons that take you to another page. That clickable element is a **hyperlink**. Without hyperlinks, the Web would not be a web at all — it would be a pile of separate documents.

DEFINITION **Hyperlink, absolute URL and relative URL**

A **hyperlink** (or simply link) is an element of a web page which, when clicked, takes the browser to another resource: another page, another website, an e-mail address, or a position inside the same page. In HTML it is created with the tag `<a>` (for anchor) and its attribute `href` (hypertext reference):

```
<a href="destination">clickable text</a>
```

The destination is a **URL** (Uniform Resource Locator). Two kinds must be distinguished:

- an **absolute URL** gives the complete address of a resource, including the protocol and the domain name, e.g. `https://www.minesec.gov.cm`;
- a **relative URL** gives the address of a file relative to the current page, e.g. `page2.html` or `images/photo.jpg`. It only works between files of the same site.

The three everyday cases. The syntax is always the same; only the value of `href` changes.

EXEMPLE **Three kinds of links**

- Absolute link (another website):

  ```
  <a href="https://www.antic.cm">Visit ANTIC</a>
  ```

- Relative link (another page of my site):

  ```
  <a href="page2.html">Go to page 2</a>
  ```

  If the target file sits in a sub-folder: `<a href="images/gallery.html">Gallery</a>`. To go up one folder level, use `../`: `<a href="../index.html">Home</a>`.

- E-mail link:

  ```
  <a href="mailto:principal@myhighschool.cm">Write to the principal</a>
  ```

  Clicking it opens the visitor's mail program with the address already filled in.

Internal anchors (a GCE-useful extra). On a long page — think of a page listing all 58 divisions of Cameroon — it is convenient to jump directly to a section. This is done in two steps: first mark the target with the attribute `id`, then link to it with `href="#name"`.

EXEMPLE **Jumping inside the same page**

`<h2 id="west">West Region</h2>` … somewhere else on the page … `<a href="#west">Jump to the West Region</a>`

The `#` sign tells the browser: "the target is an `id` on this page". You can even combine both: `<a href="page2.html#west">` jumps to the section `west` of `page2.html`.

WARNING **Links that lead nowhere**

The most frequent student errors are: forgetting the closing `</a>` tag (the whole rest of the page becomes a link), misspelling the file name in a relative link, and writing `href="C:\Users\..."` (a Windows path is not a URL). Always test every link by clicking it in the browser.

3.2 Inserting images with `<img>`

Intuition. A page about the West Region without a photo of Mount Bamboutos or of a chiefdom would be dull. Images are inserted with the `<img>` tag, which is an empty tag: it has no closing partner because the image is a reference to an external file, not content typed between two tags.

DEFINITION **The `<img>` tag and its attributes**

```
<img src="images/bafoussam.jpg" alt="Bafoussam market" width="300" height="200">
```

- `src` (source): the URL (usually relative) of the image file — compulsory;
- `alt` (alternative text): a short description of the image — compulsory in good practice;
- `width` and `height`: the display size in pixels (optional).

The common formats are **JPEG** (photographs, small files), **PNG** (drawings, logos, supports transparency) and **GIF** (simple animations).

PROPERTY **The role of the `alt` attribute**

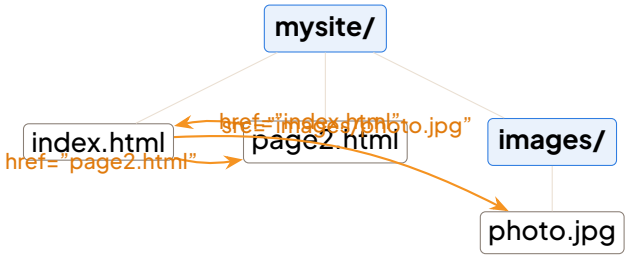
The `alt` text is displayed in place of the image when the file cannot be loaded (broken link, slow connection) and it is read aloud by screen readers used by visually impaired visitors. It also helps search engines understand the page. Omitting `alt` makes a site fragile and inaccessible. (Statement to know for the GCE; no proof required.)

WARNING **File names are case-sensitive**

On many web servers (especially Linux servers, which host most Cameroonian sites), `Photo.jpg` and `photo.jpg` are two different files. If your code says `src="photo.jpg"` but the file is saved as `Photo.jpg`, the image will appear on your Windows home computer yet be broken once published online. Adopt a strict habit: lower-case names, no spaces, no accents (write `bafoussam-market.jpg`).

3.3 Organising a small site: relative paths

A real site is a folder containing several files. Good organisation from the start saves hours of debugging: keep all images in an `images` sub-folder and always use relative paths, so that the whole folder can be copied to a USB key or a server without breaking any link.



A small site folder and the relative paths used to connect its files.

KEY POINTS **Golden rule of paths**

Write every link and every image source as a **relative path** from the current file. Then the site works identically on your computer, on a friend's computer and on the school server.

3.4 Lists: unordered, ordered, nested and description lists

Intuition. When you write "the regions of the North: Adamawa, North, Far North", a list is clearer than a sentence. HTML offers three list types.

DEFINITION **The three list types**

- Unordered list (bullet points): `<ul>` containing items `<li>`;
- Ordered list (numbered automatically): `<ol>` containing items `<li>`;
- Description list (term + explanation): `<dl>` containing terms `<dt>` and definitions `<dd>`.

EXEMPLE **The three lists in action**

Unordered:

```
<ul> <li>Garoua</li> <li>Maroua</li> <li>Ngaoundéré</li> </ul>
```

Ordered (the browser numbers 1, 2, 3 by itself):

```
<ol> <li>Fill the form</li> <li>Pay the fee</li> <li>Collect the receipt</li>
</ol>
```

Description:
<dl> <dt>HTML</dt> <dd>Language describing the structure of a web page</dd>
</dl>

Nested lists. A list can live inside an item of another list — perfect for showing regions and their divisions:

EXEMPLE **A nested list**

```
<ul>
~~<li>West Region
~~~~<ul> <li>Bamboutos</li> <li>Mifi</li> <li>Noun</li> </ul>
~~</li>
~~<li>Littoral Region</li>
</ul>
```

Notice: the inner `<ul>` is placed inside the `<li>` of the West Region, before its closing `</li>`.

WARNING **An `<li>` never lives alone**

Every `<li>` must be inside a `<ul>` or an `<ol>` — never directly in the page body. Writing `<li>Garoua</li>` without a surrounding list is invalid HTML: some browsers display it by luck, others ignore it, and you lose marks in a GCE practical for incorrect structure. Similarly, a nested list goes inside an `<li>`, never directly between two `<li>` elements.

3.5 Tables: displaying data in rows and columns

Intuition. A school timetable, the results of a mock GCE, the price list of a cybercafé in Bamenda: whenever data has rows and columns, a table is the right structure. An HTML table is built row by row, and each row is filled cell by cell.

DEFINITION **Table tags**

- `<table>` … `</table>` delimits the whole table; the attribute `border="1"` draws visible borders;
- `<tr>` (table row) delimits one row;
- `<th>` (table heading) is a header cell, displayed bold and centred;
- `<td>` (table data) is an ordinary cell;

- `<caption>` placed just after `<table>` gives the table a title;
- the attributes `colspan="n"` and `rowspan="n"` make a cell stretch over *n* columns or *n* rows.

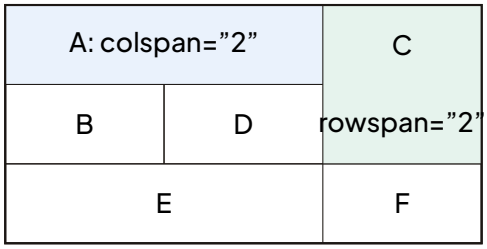
METHOD **Building a table step by step**

1. Decide the number of columns and sketch the table on paper.
2. Open `<table border="1">` and add the `<caption>`.
3. Write the first row with `<tr>` and one `<th>` per column (the headings).
4. For each following row, open `<tr>`, write one `<td>` per column, then close `</tr>`.
5. Count the cells of every row: with merging, each row must still account for the same total number of columns (`colspan="2"` counts as two).
6. Close `</table>` and check in the browser.

EXEMPLE **A small results table**

```
<table border="1">
~~<caption>Mock GCE results</caption>
~~<tr> <th>Subject</th> <th>Mark / 20</th> </tr>
~~<tr> <td>ICT</td> <td>16</td> </tr>
~~<tr> <td>Mathematics</td> <td>13</td> </tr>
</table>
```

Merging cells. Sometimes one piece of information covers several columns or rows: a teacher who takes two classes, a heading that spans “Term 1” and “Term 2”. Use `colspan` to merge horizontally and `rowspan` to merge vertically. The figure below shows the geometry of a table where cell A spans two columns and cell C spans two rows.



Cell merging: A covers two columns (`colspan`), C covers two rows (`rowspan`).

WORKED EXAMPLE 3 **Coding the merged table of the figure**

Write the HTML code of the table shown above: row 1 contains A (two columns) and C; row 2 contains B, D, with C continuing; row 3 contains E and F.

SOLUTION

Solution. Row 1: A uses `colspan="2"`, C uses `rowspan="2"`. Row 2 must not repeat C — it only contains B and D, because C already occupies the third column. Row 3 has two cells: the first spans two columns (E), the second is a single cell (F).

```
<table border="1">
~~<tr> <td colspan="2">A</td> <td rowspan="2">C</td> </tr>
~~<tr> <td>B</td> <td>D</td> </tr>
~~<tr> <td colspan="2">E</td> <td>F</td> </tr>
</table>
```

Check the column count: row 1 has 2 + 1 = 3 columns, row 2 has 1 + 1 + 1(C from above) = 3, row 3 has 2 + 1 = 3. The table is coherent.

WARNING **The classic `rowspan` trap**

After a `rowspan="2"`, the next row must contain one cell fewer, because the merged cell already occupies a position. Forgetting this shifts all the following cells and destroys the table. Always count columns per row, as in the method above.

3.6 Mini-practice: the page “My Region”

EXPERIMENT **Guided practice: a page about your region**

Create the folder `myregion/` with a sub-folder `images/` containing one photo of your region

(e.g. `images/region.jpg`). Then write `index.html` combining everything from this section:

1. a heading `<h1>My Region: West</h1>` and a short paragraph;
2. the photo: `<img src="images/region.jpg" alt="Landscape of the West Region" width="320">`;
3. an unordered list of three divisions (Bamboutos, Mifi, Noun);
4. a bordered table of two schools with columns School and Town, introduced by a `<caption>`;
5. a relative link `<a href="page2.html">Next page</a>` and a mail link `mailto:` to the school;
6. an `id` on the table's heading and a `#` link at the top of the page jumping to it.

Open the page, click every link, and rename the photo to `Region.jpg` to observe the broken image and the `alt` text appearing — a live demonstration of the case-sensitivity warning.

EXERCISE 3 **Check your mastery**

1. Write the tag for a link to `https://www.camtel.cm` displaying the text "CAMTEL".
2. Your page is in `mysite/pages/info.html` and the image is `mysite/images/logo.png`. Write the correct `<img>` tag (hint: use `../`).
3. Correct this code: `<ul> <li>Yaoundé </ul> <li>Douala</li> </ul>`.
4. Build a 3 × 3 table of term averages where the first cell (top-left) spans the first two columns.

SOLUTION **Exercise 4**

1. `<a href="https://www.camtel.cm">CAMTEL</a>`
2. `<img src="../images/logo.png" alt="School logo">` — we go up one level with `../` then down into `images`.
3. The first `<li>` is not closed and the list is closed too early. Correct version: `<ul> <li>Yaoundé</li> <li>Douala</li> </ul>`.
4. `<table border="1"> <tr> <td colspan="2">Averages</td> <td>Rank</td> </tr>` then two rows of three `<td>` each, then `</table>`. The first row accounts for 2 + 1 = 3

columns, like the others.

KEY POINTS **What you must remember**

- `<a href="...">` creates links: absolute URLs for other sites, relative URLs inside your site, `mailto:` for e-mail, `#id` for internal anchors.
- `<img src="..." alt="...">` inserts an image; `alt` is essential for accessibility and broken files; file names are case-sensitive.
- Lists: `<ul>`, `<ol>`, `<dl>`; every `<li>` lives inside a list.
- Tables are built row by row: `<table>`, `<tr>`, `<th>`, `<td>`, `<caption>`, with `colspan`/`rowspan` for merging — always count your columns.
- Organise your site in folders and use relative paths everywhere.

You are now ready for the last section of this chapter: collecting user input with forms and assembling everything in an integration project.

4 FORMS, VALIDATION AND INTEGRATION PROJECT

In the previous section we learned how to link pages together, embed images, organise information in lists and present data in tables. We now have all the structural building blocks for a small website. The final piece is interactivity: allowing the visitor to send information to the site — a search query, a registration, a message, an order. This is the role of **HTML forms**. After mastering forms we will see how to validate our markup, touch briefly on styling with CSS, and then put everything together in a guided integration project (Lesson 45).

4.1 Web forms — collecting user data

DEFINITION **Web form**

A **web form** is a section of an HTML document that contains interactive controls (text fields, buttons, menus, etc.) allowing the user to enter data. The data is sent to a server when the user submits the form. In HTML the form is delimited by the `<form>` element.

The `<form>` element is a container. Its two most important attributes are:

- `action` — the URL of the server-side script that will process the data (e.g. `action="process.php"`). At GCE level we only need to know that this attribute exists; writing the server script is beyond the syllabus.

- `method` — the HTTP method used to send the data: `get` (data appended to the URL, visible, limited length) or `post` (data sent in the request body, not visible in the URL, no practical length limit). For sensitive data (passwords, personal details) `post` is preferred.

Aspect	GET	POST
Data visibility	In URL (query string)	In request body (hidden)
Length limit	Yes (browser-dependent, 2000 chars)	Practically none
Bookmarkable / shareable	Yes	No
Suitable for	Search queries, filters	Login, registration, payments

EXEMPLE **Minimal form skeleton**

```
<form action="register.php" method="post">
 <!-- form controls go here -->
</form>
```

4.2 Common form controls

All visible controls are placed inside the `<form>` tags. Each control that should send a value to the server **must have a `name` attribute**.

DEFINITION **The `name` attribute**

The `name` attribute gives the control a unique identifier (within the form) so that the server can recognise which value corresponds to which field. Without `name`, the control's value is not submitted.

PROPERTY **Radio buttons vs. checkboxes**

- Radio buttons (`<input type="radio">`) with the same *`name`* form a mutually exclusive group: only one can be selected at a time.
- Checkboxes (`<input type="checkbox">`) are independent; each can be checked or unchecked regardless of the others. If several checkboxes share the same `name`, the server receives multiple values for that name (often as an array).

Control	HTML snippet	Purpose
Text field	`<input type="text" name="username">`	Single-line text (name, email, etc.)
Password	`<input type="password" name="pwd">`	Masked input (bullets)
Email	`<input type="email" name="email">`	Email with built-in browser validation
Radio button	`<input type="radio" name="gender" value="M"> Male`	Choose one from a set
Checkbox	`<input type="checkbox" name="sports" value="football"> Football`	Choose zero or more
Submit button	`<input type="submit" value="Register">`	Sends the form data
Reset button	`<input type="reset" value="Clear">`	Resets all controls to initial values
Multi-line text	`<textarea name="address" rows="4" cols="40"></textarea>`	Longer text (address, comments)
Drop-down list	`<select name="class"><option value="L6A">Lower Sixth A</option>...</select>`	Choose one from a list

WARNING **The `name` attribute is mandatory for submission**

If you write `<input type="text">` without `name`, the user can type in it but the value will never reach the server. This is a very common mistake in exams.

WARNING **HTML alone cannot process form data**

The `<form>` element only collects and sends data. To actually store it in a database, send an email, or compute a result, you need a server-side technology (PHP, Python, Node.js, etc.). This is outside the GCE ICT syllabus but you must be aware of the distinction.

4.3 Grouping controls — `<fieldset>` and `<legend>`

For accessibility and visual clarity, related controls can be grouped.

EXEMPLE **Fieldset example**

```
<fieldset>
 <legend>Gender</legend>
 <label><input type="radio" name="gender" value="M">Male</label>
 <label><input type="radio" name="gender" value="F">Female</label>
</fieldset>
```

The `<label>` element improves usability: clicking the label text focuses the associated control. Two ways to associate:

- Wrap the control: `<label>Name: <input type="text" name="name"></label>`
- Use `for`/`id`: `<label for="email">Email:</label> <input type="email" id="email" name="email">`

4.4 Worked example — a registration form

WORKED EXAMPLE 4 **Registration form for a school club**

Write the HTML for a form that collects:

1. Full name (text, required)
2. Email (email type, required)
3. Gender (radio buttons: Male, Female, Other)
4. Class (drop-down: Form 1, Form 2, …, Upper Sixth)
5. Sports interested in (checkboxes: Football, Basketball, Athletics, Other)
6. A short bio (textarea)
7. Submit and Reset buttons

The form should send data to `club_register.php` using POST.

SOLUTION

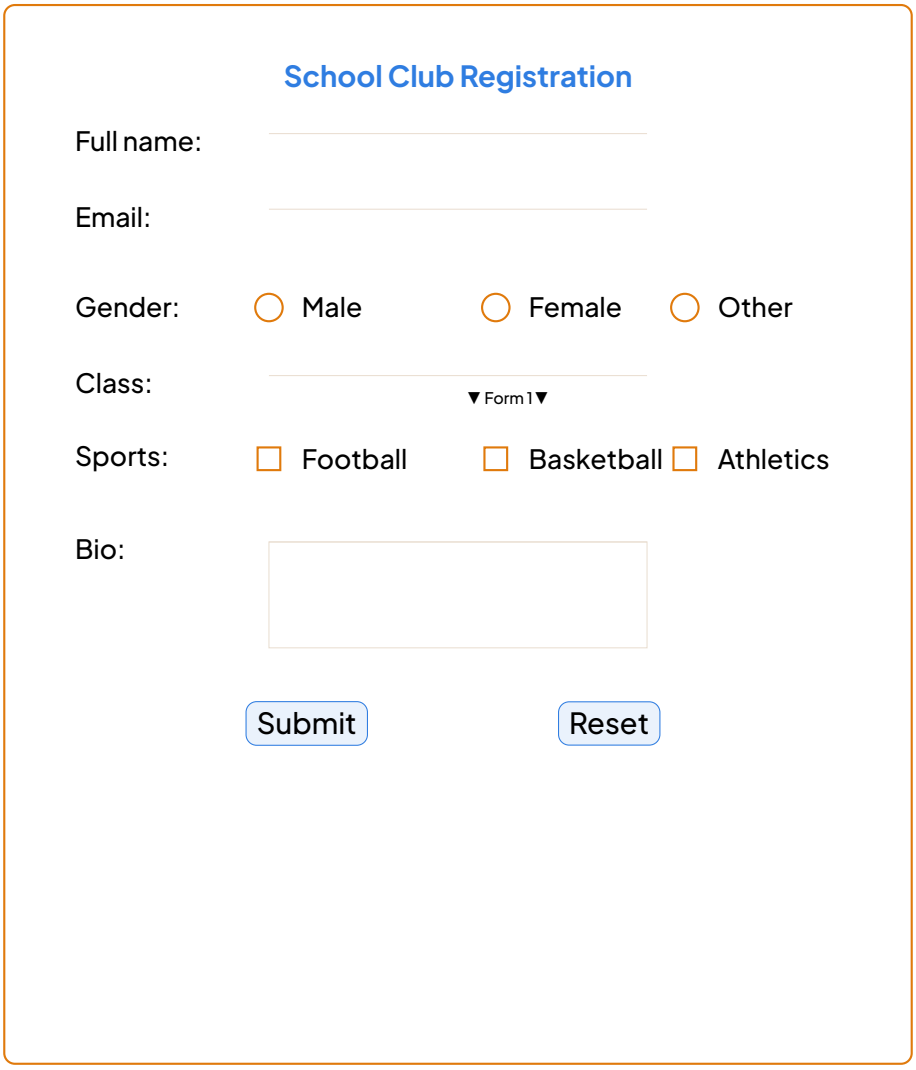
Solution:

```
<form action="club_register.php" method="post">
 <fieldset>
 <legend>Personal Information</legend>
 <p>
 <label for="fullname">Full name:</label>
 <input type="text" id="fullname" name="fullname" required>
 </p>
 <p>
 <label for="email">Email:</label>
 <input type="email" id="email" name="email" required>
 </p>
 </fieldset>

 <fieldset>
 <legend>Gender</legend>
 <label><input type="radio" name="gender" value="M" required>Male</label>
 <label><input type="radio" name="gender" value="F">Female</label>
 <label><input type="radio" name="gender" value="O">Other</label>
 </fieldset>

 <fieldset>
 <legend>Class</legend>
 <label for="class">Select class:</label>
 <select id="class" name="class" required>
 <option value="">-- Choose --</option>
 <option value="F1">Form 1</option>
 <option value="F2">Form 2</option>
 <option value="F3">Form 3</option>
 <option value="F4">Form 4</option>
 <option value="F5">Form 5</option>
 <option value="L6">Lower Sixth</option>
 <option value="U6">Upper Sixth</option>
 </select>
 </fieldset>

 <fieldset>
 <legend>Sports</legend>
 <label><input type="checkbox" name="sports" value="football">Football</label>
 <label><input type="checkbox" name="sports" value="basketball">Basketball</label>
 <label><input type="checkbox" name="sports" value="athletics">Athletics</label>
```



Mock-up of the registration form (visual representation only).

4.5 Validating your HTML — writing correct markup

Browsers are forgiving: they try to display a page even if the HTML contains errors. But invalid markup leads to unpredictable rendering, accessibility problems, and maintenance nightmares. As a developer you must validate your code.

KEY POINTS **Common validation checks**

- All tags closed: `<p>Paragraph</p>` not `<p>Paragraph`. Void elements (`<img>`, `<br>`, `<input>`) are self-closing in XHTML (`<br />`) but in HTML5 the slash is optional.
- Proper nesting: tags must close in the reverse order they opened. `<b><i>text</i></b>` is correct; `<b><i>text</b></i>` is invalid.

- Attribute values quoted: `<img src="logo.png" alt="Logo">` — double quotes are standard; single quotes work but be consistent.
- Required attributes present: `alt` on `<img>`, `name` on form controls, `href` on `<a>`.
- Unique `id` values: no two elements may share the same `id`.

WARNING **Bad nesting example**

`<p><b>Bold and <i>italic</b> only</i></p>` — the `<b>` closes before the `<i>`, so the browser must guess. Always close inner tags first: `<p><b>Bold and <i>italic</i></b> only</p>`.

METHOD **Validating a page — step by step**

1. Open the page in a browser.
2. Right-click → View Page Source (or press `Ctrl+U`) to see the raw HTML the browser received.
3. Look for red flags: unclosed tags, mismatched nesting, missing quotes.
4. (Optional but recommended) Copy the source and paste it into the W3C Markup Validation Service (`https://validator.w3.org/`) — it gives a detailed error report with line numbers.
5. Fix errors in your editor, save, refresh the browser, repeat until the validator shows "Document checking completed. No errors or warnings to show."

4.6 A glimpse of CSS — separating structure from presentation

HTML describes what the content is (heading, paragraph, table). CSS (Cascading Style Sheets) describes how it looks (colours, fonts, layout). The GCE syllabus does not require you to write CSS, but you should know it exists and recognise an inline style — the simplest (and least maintainable) way to apply CSS.

EXEMPLE **Inline style example**

```
<p style="color:blue; font-family:Arial;">This paragraph is blue and in Arial.</p>
<h1 style="text-align:center; background-color:#f0f0f0;">Centered heading with grey background
```

DID YOU KNOW? **Why not use inline styles everywhere?**

Inline styles mix presentation with structure, making the HTML hard to read and maintain. In real projects we use external style sheets (`<link rel="stylesheet" href="style.css">`) so one CSS file controls the look of the whole site. This is covered in university-level web development courses.

4.7 Planning a mini-website — from paper to code

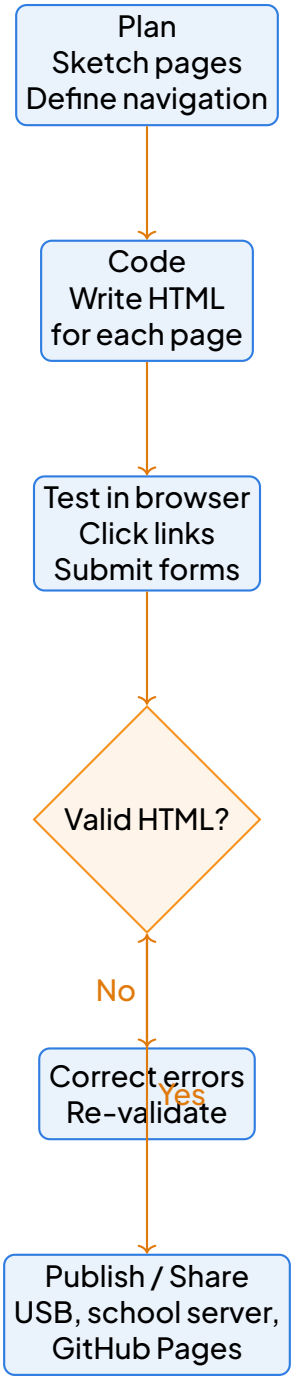
Before writing a single tag, a professional developer plans. The syllabus (Lesson 45) expects you to go through a complete cycle for a small site (2–3 pages).

METHOD **Plan–Code–Test–Correct cycle for a mini-website**

1. Sketch on paper: Draw each page as a wireframe — header, navigation, main content, footer. Decide what each page will contain (text, images, form, table).
2. Define navigation: List the links between pages (e.g. Home → About → Contact → Home). Ensure every page is reachable.
3. Create a folder structure:

```
mysite/
  index.html
  about.html
  contact.html
  images/
    logo.png
    photo.jpg
  style.css (optional, for extra credit)
```

4. Code each page: Start with the common skeleton (`<!DOCTYPE html>`, `<html>`, `<head>`, `<body>`), then add the specific content. Reuse the navigation menu on every page.
5. Test in browser: Open `index.html`. Click every link. Fill and submit the form (check the URL to see if `GET` works). Resize the window.
6. Validate: Use View Source and/or the W3C validator. Fix every error.
7. Peer review: Exchange with a classmate. Use the checklist below.
8. Publish/share: Copy the folder to a USB drive, school server, or free hosting (GitHub Pages, Netlify) to show the live site.



Project workflow: plan → code → test → validate → correct → publish.

4.8 Integration project (Lesson 45) — guided activity

Scenario: Build a 3-page website for "Buea Tech Hub" (a fictional local co-working space) or for your own school.

EXPERIMENT **Project brief — Buea Tech Hub**

Pages required:

1. `index.html` — Home: logo, tagline, brief intro, navigation menu (Home | Services | Contact), photo of the hub.
2. `services.html` — Services: table of offers (Hot desk, Dedicated desk, Meeting room, Event space) with columns: Service, Capacity, Price (FCFA/day), Amenities. Use `<thead>`, `<tbody>`, `<th scope="col">`.
3. `contact.html` — Contact: form with name, email, phone, subject (dropdown: General, Booking, Partnership, Other), message (textarea), submit/reset. Footer with address, phone, social media links (icons as images with `alt` text).

Technical checklist (peer review):

Item	Check
Doctype	`<!DOCTYPE html>` on every page
Character set	`<meta charset="UTF-8">` in `<head>`
Title	Unique, descriptive `<title>` per page
Navigation	Identical menu on all pages; links work (relative paths)
Images	`alt` attributes present; files in `images/` folder
Table	`<thead>`, `<tbody>`, `scope` on `<th>`
Form	`action`, `method="post"`; every control has `name`; `<label>` used
Nesting	No overlapping tags (validate with W3C)
Quotes	All attribute values in double quotes
IDs unique	No duplicate `id` values

KEY POINTS **Key take-aways for the exam**

- A form is created with `<form action="..." method="post|get">`. Controls need `name` to be submitted.
- Radio buttons share a `name` for single choice; checkboxes are independent (or share a name for multiple values).
- `<fieldset>` + `<legend>` group related controls; `<label>` improves accessibility.
- Validation: close all tags, nest properly, quote attributes, provide required attributes (`alt`, `name`, etc.).
- CSS exists to style pages; inline style uses `style="property:value"`.

- The development cycle: plan → code → test → validate → correct → publish.
- HTML forms only send data; processing requires a server-side language (PHP, etc.) — not examined at GCE.

EXERCISE 4 **Practice questions**

1. Write the HTML for a `<form>` that sends a search query to `search.php` using GET. The form should have a text input named `q` and a submit button labelled "Search".
2. Explain why the following snippet is invalid and correct it: `<p><b>Welcome to <i>our site</b></i></p>`
3. A form has three checkboxes all with `name="colour"` and values `red`, `green`, `blue`. The user checks red and blue. What data is sent to the server? (Assume `method="get"`.)
4. List three things the W3C validator would flag as errors in this fragment:

   ```
   <img src=logo.png alt=Logo>
   <input type=text name=username>
   <a href=about.html>About</a>
   ```

5. Sketch (on paper) the wireframe for a 2-page site for a school canteen: a menu page (table of dishes and prices) and an order page (form). Indicate the navigation link(s).

SOLUTION **Exercise 1**

```
<form action="search.php" method="get">
 <label for="q">Search:</label>
 <input type="text" id="q" name="q">
 <input type="submit" value="Search">
</form>
```

SOLUTION **Exercise 2**

The tags are improperly nested: `<b>` opens, then `<i>`, but `</b>` closes before `</i>`. Correct nesting: `<p><b>Welcome to <i>our site</i></b></p>`

SOLUTION **Exercise 3**

With `method="get"`, the URL will be something like: `search.php?colour=red&colour=blue` (The `name` "colour" appears twice, once for each checked value.)

SOLUTION **Exercise 4**

1. `<img src=logo.png alt=Logo>` — attribute values not quoted.
2. `<input type=text name=username>` — attribute values not quoted; also missing closing slash if XHTML, but in HTML5 it's void.
3. `<a href=about.html>About</a>` — attribute value not quoted.

(Also, the `<img>` lacks a closing slash in XHTML, but HTML5 allows it. The validator in HTML5 mode may only warn about missing quotes.)

SOLUTION **Exercise 5**

(Wireframe description — no unique answer.) Page 1 (menu.html): Header (canteen name), Nav (Menu | Order), Table (Dish | Price), Footer. Page 2 (order.html): Header, Nav, Form (Name, Class, Dish dropdown, Quantity, Submit), Footer. Navigation links: menu.html ↔ order.html.

With forms, validation, a touch of CSS awareness, and the complete plan–code–test–correct cycle, you now have every skill required by the GCE Advanced Level ICT syllabus for "Develop simple web pages". The integration project is your chance to prove it — build something real, validate it rigorously, and show it proudly. Good luck!

5 INTEGRATION ACTIVITY

5.1 Aims of the activity

- Consolidate all HTML skills acquired in this chapter by designing, coding and testing a complete multi-page website.
- Practise the **plan–code–test–correct** development cycle in a collaborative team setting.
- Produce a valid, well-structured site that meets a real-world specification for a Cameroonian school.
- Develop evaluation skills through peer assessment using a detailed checklist.

5.2 Situation

Context. The administration of Collège Bilingue de la Jeunesse (CBJ), a bilingual secondary school located in the Mfoundi division of Yaoundé, wishes to establish an online presence. The principal has invited the Upper Sixth ICT class to submit a prototype website that could later be adopted as the official school site. The prototype must present the school's identity, provide essential information to parents and students, and allow visitors to send enquiries.

Your team. You work in groups of 3–4 students. Each team acts as a junior web development agency. You have two weeks (four double periods) to deliver the prototype.

Technical requirements.

1. The site must consist of exactly four HTML pages linked by a consistent navigation menu:
   - `index.html` – Home page
   - `about.html` – About us
   - `timetable.html` – Time-table / Results
   - `contact.html` – Contact form
2. All pages must use **relative hyperlinks** (e.g. `<a href="about.html">`) and share the same header, navigation bar and footer structure.
3. The home page must contain:
   - A level-1 heading with the school name.
   - A welcome paragraph (at least 3 sentences) describing the school's mission.
   - The school logo (image file `logo.png` placed in an `images` folder) with an appropriate `alt` attribute.
4. The About us page must contain:
   - A level-1 heading "About Collège Bilingue de la Jeunesse".
   - Two formatted paragraphs (use `<p>`, `<strong>`, `<em>` where appropriate) presenting the school's history and values.

- An unordered list of at least five clubs/associations (e.g. “Science Club”, “Debate Club”, “Environmental Club”, “Music & Dance”, “ICT Club”).

5. The Time-table / Results page must contain:
   - A level-1 heading “Time-table & Examination Results”.
   - A properly structured HTML table showing a sample weekly timetable for Form 5 (Monday–Friday, 8 periods per day). The table must have a `<caption>`, `<thead>`, `<tbody>`, `<th>` (with `scope` attributes) and `<td>` cells.
6. The Contact page must contain:
   - A level-1 heading “Contact Us”.
   - A form with the following controls, each correctly associated with a `<label>`:
     - Text input for “Full Name” (required).
     - Select dropdown for “Class” (options: Form 1 – Upper Sixth).
     - Textarea for “Message” (required, minimum 20 characters).
     - Submit button labelled “Send Enquiry”.
   - The form must use `method="post"` and `action="process.php"` (the back-end script is not required for this prototype).
7. All pages must declare the HTML5 doctype, specify `lang="en"`, include a `<meta charset="UTF-8">` and a responsive viewport meta tag.
8. CSS styling is not required for this activity; the focus is on **semantic HTML** structure and validity.

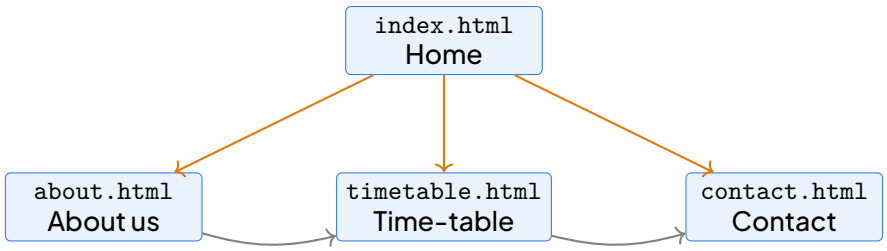
Deliverables.

- A folder named `CBJ_Website` containing the four `.html` files and an `images` subfolder with `logo.png` (any placeholder image of your choice).
- A one-page team report (handwritten or typed) describing the division of work, difficulties encountered and how they were resolved.
- A completed peer-evaluation checklist (provided by the teacher) for another team’s site.

5.3 Tasks

Task 1 – Planning (30 minutes).

1. As a team, sketch a site map on paper showing the four pages and the navigation links between them.
2. Assign roles: HTML coder, Content writer, Tester/Validator, Coordinator. Write the names next to each role in your report.
3. List the exact file and folder structure you will create (e.g. `CBJ_Website/index.html`, `CBJ_Website/images/` …).



Site map of the CBJ prototype: every page is reachable from the navigation menu.

Task 2 – Coding the skeleton (45 minutes).

1. Create the folder structure on your workstation.
2. In each HTML file, write the complete HTML5 boilerplate (doctype, `html`, `head`, `body`). Include the common `<header>`, `<nav>` (with an unordered list of links to the four pages) and `<footer>` (with a copyright notice "© 2025 Collège Bilingue de la Jeunesse").
3. Verify that all relative links in the navigation menu point to the correct filenames.

Task 3 – Implementing page-specific content (90 minutes).

1. **Home page:** Insert the heading, welcome paragraph and logo image. Ensure the image path is `images/logo.png` and the `alt` text reads "CBJ logo – a book and a torch".
2. **About page:** Write the two paragraphs and the unordered list of clubs. Use `<strong>` for the school's founding year (1998) and `<em>` for the motto "Excellence through Discipline".
3. **Time-table page:** Construct the table. The first row of `<thead>` should contain "Period" and the five weekdays. Each subsequent row in `<tbody>` represents a period (1–8) with the subject taught that day. Use `scope="col"` for column headers and `scope="row"` for the period numbers.
4. **Contact page:** Build the form with the required fields. Add the `required` attribute to the name and message fields. Set `minlength="20"` on the textarea. Use `<label for="...">` matching each input's `id`.

Task 4 – Testing and validation (45 minutes).

1. Open each page in a browser (Chrome/Firefox). Check that:
   - All navigation links work and no "404 Not Found" errors appear.
   - The logo displays, and the alt text appears when the image is broken (temporarily rename `logo.png` to test).
   - The table renders with headers and data aligned.
   - The form submits (you will see a `process.php` "not found" page – that is expected, since no back-end script exists).
2. Validate each HTML file using the W3C Markup Validation Service (`https://validator.w3.org/`) via "Validate by File Upload". Record any errors and correct them.

3. Complete the peer-evaluation checklist for another team's site (the teacher will distribute the sites on a shared drive).

Task 5 – Presentation and reflection (30 minutes).

1. Each team presents their site (2 minutes) highlighting the structure, any semantic choices made and how they handled validation errors.
2. Submit the folder, team report and completed checklist to the teacher.

5.4 Model answer

1. Folder structure

```
CBJ_Website/
|-- index.html
|-- about.html
|-- timetable.html
|-- contact.html
`-- images/
~~~~`-- logo.png
```

2. Common boilerplate (repeated in each file with a page-specific title)

```
<!DOCTYPE html>
<html lang="en">
<head>
~~<meta charset="UTF-8">
~~<meta name="viewport" content="width=device-width, initial-scale=1.0">
~~<title>College Bilingue de la Jeunesse -- Home</title>
</head>
<body>
~~<header>
~~~~<h1>College Bilingue de la Jeunesse</h1>
~~</header>
~~<nav>
~~~~<ul>
~~~~~~<li><a href="index.html">Home</a></li>
~~~~~~<li><a href="about.html">About Us</a></li>
~~~~~~<li><a href="timetable.html">Time-table / Results</a></li>
~~~~~~<li><a href="contact.html">Contact</a></li>
~~~~</ul>
~~</nav>
~~<main>
~~~~<!-- Page-specific content goes here -->
~~</main>
~~<footer>
~~~~<p>&copy; 2025 College Bilingue de la Jeunesse</p>
~~</footer>
</body>
</html>
```

3. index.html – Home page (main content)

```
<section>
~~<h2>Welcome to CBJ</h2>
~~<p>College Bilingue de la Jeunesse is a leading bilingual secondary school
~~~~~in Yaounde, committed to academic excellence and moral formation.  Our
~~~~~dedicated staff nurture every student's potential in a supportive,
~~~~~multicultural environment.</p>
~~<p>We offer both the Anglophone and Francophone curricula, preparing
~~~~~learners for the GCE Advanced Level and the Baccalaureat respectively.</p>
~~<img src="images/logo.png" alt="CBJ logo -- a book and a torch"
~~~~~~~width="150" height="150">
</section>
```

4. about.html – About us (main content)

```
<section>
~~<h2>About College Bilingue de la Jeunesse</h2>
~~<p>Founded in <strong>1998</strong>, CBJ has grown from a modest
~~~~~establishment into a reference institution in the Mfoundi division.
~~~~~Our motto, <em>Excellence through Discipline</em>, guides every
~~~~~pedagogical decision.</p>
~~<p>We pride ourselves on a holistic education that balances academics,
~~~~~sports and extracurricular activities.</p>
~~<h3>Clubs and Associations</h3>
~~<ul>
~~~~<li>Science Club</li>
~~~~<li>Debate Club</li>
~~~~<li>Environmental Club</li>
~~~~<li>Music &amp;amp; Dance</li>
~~~~<li>ICT Club</li>
~~</ul>
</section>
```

WARNING **Escaping special characters in HTML**

Inside HTML code, the ampersand must be written as the entity `&` (e.g. `Music & Dance`) and the copyright sign as `©`. A raw `&` in the source is a validation error. Likewise use `<` and `>` for < and > when they are text, not tags.

5. timetable.html – Time-table / Results (main content)

```
<section>
~~<h2>Time-table &amp; Examination Results</h2>
~~<table>
~~~~<caption>Sample Weekly Timetable -- Form 5 (Arts)</caption>
~~~~<thead>
~~~~~~<tr>
~~~~~~~~<th scope="col">Period</th>
~~~~~~~~<th scope="col">Monday</th>
~~~~~~~~<th scope="col">Tuesday</th>
~~~~~~~~<th scope="col">Wednesday</th>
~~~~~~~~<th scope="col">Thursday</th>
~~~~~~~~<th scope="col">Friday</th>
~~~~~~</tr>
~~~~</thead>
~~~~<tbody>
~~~~~~<tr><th scope="row">1</th><td>English Language</td><td>Mathematics</td>
~~~~~~~~~~<td>History</td><td>Geography</td><td>English Literature</td></tr>
~~~~~~<tr><th scope="row">2</th><td>Mathematics</td><td>English Language</td>
~~~~~~~~~~<td>Geography</td><td>History</td><td>Mathematics</td></tr>
~~~~~~<tr><th scope="row">3</th><td>History</td><td>Geography</td>
~~~~~~~~~~<td>English Language</td><td>Mathematics</td><td>Civic Education</td></tr>
~~~~~~<tr><th scope="row">4</th><td>Geography</td><td>History</td>
~~~~~~~~~~<td>Mathematics</td><td>English Language</td><td>Computer Science</td></tr>
~~~~~~<tr><th scope="row">5</th><td>Computer Science</td><td>Civic Education</td>
~~~~~~~~~~<td>English Literature</td><td>Physics</td><td>English Language</td></tr>
~~~~~~<tr><th scope="row">6</th><td>Physics</td><td>Computer Science</td>
~~~~~~~~~~<td>Civic Education</td><td>English Literature</td><td>History</td></tr>
~~~~~~<tr><th scope="row">7</th><td>Civic Education</td><td>Physics</td>
~~~~~~~~~~<td>Computer Science</td><td>Biology</td><td>Geography</td></tr>
~~~~~~<tr><th scope="row">8</th><td>Sports</td><td>Sports</td>
~~~~~~~~~~<td>Club Activities</td><td>Club Activities</td><td>Guidance</td></tr>
~~~~</tbody>
~~</table>
</section>
```

6. contact.html – Contact page (main content)

```
<section>
~~<h2>Contact Us</h2>
~~<form action="process.php" method="post">
~~~~<div>
~~~~~~<label for="fullname">Full Name:</label>
~~~~~~<input type="text" id="fullname" name="fullname" required>
~~~~</div>
~~~~<div>
~~~~~~<label for="class">Class:</label>
~~~~~~<select id="class" name="class">
~~~~~~~~<option value="">-- Select --</option>
~~~~~~~~<option value="form1">Form 1</option>
~~~~~~~~<option value="form2">Form 2</option>
~~~~~~~~<option value="form3">Form 3</option>
~~~~~~~~<option value="form4">Form 4</option>
~~~~~~~~<option value="form5">Form 5</option>
~~~~~~~~<option value="lower6">Lower Sixth</option>
~~~~~~~~<option value="upper6">Upper Sixth</option>
~~~~~~</select>
~~~~</div>
~~~~<div>
~~~~~~<label for="message">Message:</label>
~~~~~~<textarea id="message" name="message" rows="5" cols="40"
~~~~~~~~~~~~~~~~required minlength="20"></textarea>
~~~~</div>
~~~~<div>
~~~~~~<button type="submit">Send Enquiry</button>
~~~~</div>
~~</form>
</section>
```

7. Validation and testing notes

- All four pages pass the W3C validator with zero errors when the `logo.png` file exists.
- The navigation menu works correctly because each `href` matches the exact filename (file names are case-sensitive on most web servers).
- The table uses semantic elements (`<caption>`, `<thead>`, `<tbody>`, `scope`), ensuring accessibility for screen readers.
- The form controls are explicitly associated with labels via `for`/`id` pairs, and client-side validation (`required`, `minlength`) is present.

8. Sample team report excerpt

Team: "Web Pioneers" – Members: Njie Samuel (Coordinator), Awa Marie (HTML Coder), Tchioma Paul (Content Writer), Fon Grace (Tester).
Division of work: Samuel created the folder structure and boilerplate; Marie coded the navigation and all four pages; Paul wrote the welcome text, history paragraphs and club list; Grace validated each page, tested links and filled the peer checklist.
Difficulties: Initially the logo did not appear because the `images` folder was placed one level too high. Corrected by moving `logo.png` into `CBJ_Website/images/`. The table headers lacked `scope` attributes; added after a validator warning.
Resolution: Used relative paths consistently, ran the validator after each page, and tested in both Chrome and Firefox.

9. Peer-evaluation checklist (teacher's version)

Criterion	Yes / No	Comments
HTML5 doctype and `lang` attribute present		
`<meta charset="UTF-8">` and viewport meta tag		
Consistent header, nav, footer on all pages		
Navigation links work (no 404 errors)		
Home page: heading, paragraph, logo with alt text		
About page: formatted paragraphs, list of at least 5 clubs		
Timetable page: table with caption, thead, tbody, th scope		
Contact page: form fields present; labels linked to controls		
Form uses `method="post"` and `action="process.php"`		
All pages validate at `https://validator.w3.org/`		

KEY POINTS **Key take-aways**

- A real-world web project requires planning, semantic HTML, rigorous testing and teamwork.
- Relative links, correct image paths and W3C validation are essential for a maintainable site.
- Accessibility (alt text, label–input association, table scopes) is not optional – it is examined.
- The plan–code–test–correct cycle mirrors professional practice and prepares you for the GCE practical paper.

WARNING **Common pitfalls**

- Forgetting the `alt` attribute on images – marks are lost for accessibility.
- Using absolute paths (e.g. `C:/Users/.../logo.png`) which break when the site is moved to a server.
- Omitting `scope` on `<th>` elements in data tables.
- Mismatched `for`/`id` values in forms – clicking the label will not focus the control.
- Writing a raw `&` in HTML text instead of the entity `&`.
- Skipping validation – easy marks lost for trivial syntax errors such as unclosed tags.

6 METHODS AND WORKED EXAMPLES

7 DEVELOP SIMPLE WEB PAGES

7.1 The Web and the Structure of an HTML Document

DEFINITION **World Wide Web and HTML**

The World Wide Web (WWW) is an information system on the Internet where documents (web pages) are identified by URLs, interlinked by hyperlinks, and accessed via HTTP/HTTPS. HTML (HyperText Markup Language) is the standard markup language for creating these documents. HTML5 is the current living standard maintained by the WHATWG and W3C.

PROPERTY **Anatomy of a Valid HTML5 Document**

A standards-compliant HTML5 document must contain, in order:

1. `<!DOCTYPE html>` — triggers Standards Mode (no quirks).
2. `<html lang="...">` — root element; `lang` declares the human language (e.g., `en`, `fr`) for accessibility and SEO.
3. `<head>` — metadata container (not rendered visually). Mandatory children:
   - `<meta charset="UTF-8">` — character encoding (supports accents, symbols, emojis).
   - `<title>Page Title</title>` — shown in browser tab, bookmarks, search results.

   Strongly recommended: `<meta name="viewport" content="width=device-width, initial-scale=1.0">` (responsive design), `<meta name="description" content="...">` (SEO), `<link rel="stylesheet" href="...">` (CSS).
4. `<body>` — all visible content. Should use semantic landmarks: `<header>`, `<nav>`, `<main>`, `<section>`, `<article>`, `<aside>`, `<footer>`.

METHOD **Creating a Valid HTML5 Document Structure**

Objective: Write the skeleton of a standard-compliant HTML5 page.

1. Declare the doctype: Write `<!DOCTYPE html>` as the very first line. It tells the browser to use Standards Mode.
2. Open the root element: `<html lang="en">` (use `"fr"` for French pages). The `lang` attribute helps accessibility and search engines.
3. Build the `<head>`: This contains metadata, not visible content.
   - `<meta charset="UTF-8">` — mandatory for proper character encoding (accents, symbols).
   - `<meta name="viewport" content="width=device-width, initial-scale=1.0">` — essential for responsive design on mobile phones (very common in Cameroon).
   - `<title>Page Title</title>` — shown in the browser tab; crucial for SEO.
   - Link CSS: `<link rel="stylesheet" href="styles.css">`.
4. Build the `<body>`: All visible content goes here. Use semantic tags: `<header>`, `<nav>`, `<main>`, `<section>`, `<article>`, `<aside>`, `<footer>`.
5. Close tags properly: Every opening tag must have a closing tag (except void elements like `<meta>`, `<img>`, `<br>`, `<hr>`, `<input>`).
6. Validate: Use the W3C Validator (https://validator.w3.org/) to catch syntax errors before the exam.

KEY POINTS **Exam Reflex**

In the GCE practical, always start by typing the skeleton perfectly. A missing `<meta charset="UTF-8">` or `<title>` loses easy marks.

WORKED EXAMPLE 5 **Building the Skeleton for "Buea Tech Hub" Homepage**

Statement: Write the complete HTML5 skeleton for the homepage of a fictional tech hub in Buea. The page title must be "Buea Tech Hub — Innovation in Silicon Mountain". Include a link to an external stylesheet `style.css` and ensure it is mobile-friendly. Do not write any visible content inside `<body>` yet.

SOLUTION

Detailed Solution:
We follow the 6-step method strictly.

1. Doctype: `<!DOCTYPE html>`

2. Root element: `<html lang="en">` (English context).
3. Head section:
   - Charset: `<meta charset="UTF-8">`
   - Viewport: `<meta name="viewport" content="width=device-width, initial-scale=1.0">`
   - Title: `<title>Buea Tech Hub -Innovation in Silicon Mountain</title>`
   - CSS Link: `<link rel="stylesheet" href="style.css">`
4. Body section: We create the semantic containers but leave them empty as requested.
5. Closing tags: Close `</body>`, then `</html>`.

Complete Code:

```
<!DOCTYPE html>
<html lang="en">
 <head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <title>Buea Tech Hub -Innovation in Silicon Mountain</title>
 <link rel="stylesheet" href="style.css">
 </head>
 <body>
 <header></header>
 <nav></nav>
 <main>
 <section></section>
 </main>
 <footer></footer>
 </body>
</html>
```

Examiner's Comments:

- The `lang="en"` attribute is a GCE favourite for accessibility marks.
- The viewport meta tag is now standard requirement for "mobile-friendly" questions.
- Indentation (2 spaces) is not required by browsers but is highly recommended in practical exams for readability and maintenance marks.

- Note the use of an em dash (—) in the title; UTF-8 handles it correctly.

DID YOU KNOW? **Cameroon Context: Silicon Mountain**

Buea, at the foot of Mount Cameroon, is nicknamed "Silicon Mountain" due to its growing tech ecosystem (incubators like ActivSpaces, startups in fintech, agritech, edtech). Building a site for a "Buea Tech Hub" is a realistic scenario you might meet in an integration project.

7.2 Formatting Text and Structuring Content

DEFINITION **Semantic HTML**

Semantic HTML means choosing tags that describe the meaning of content, not just its appearance. Example: `<strong>` means "important", not just "bold". Search engines (Google, Bing), screen readers (NVDA, JAWS, VoiceOver), and reader modes rely on semantics.

METHOD **Formatting Text and Structuring Content with Semantic Tags**

Objective: Use the correct HTML tags to give meaning (semantics) to text content, not just visual appearance.

1. Headings hierarchy: Use `<h1>` once per page (main title), then `<h2>` for major sections, `<h3>` for subsections, etc. Never skip levels (e.g., `h1` to `h3`) for SEO and screen readers.
2. Paragraphs: Wrap text blocks in `<p>`. Do not use `<br>` to separate paragraphs; use CSS margins.
3. Inline semantics:
   - `<strong>` — Important (bold by default). Use for warnings, key terms.
   - `<em>` — Stress emphasis (italic by default). Changes sentence meaning.
   - `<mark>` — Highlighted text (search results, current step).
   - `<cite>` — Title of a work (book, report, specification).
   - `<code>`, `<kbd>`, `<samp>`, `<var>` — For computer code, keyboard input, sample output, variables.
   - `<time datetime="YYYY-MM-DD">` — Machine-readable date/time (ISO 8601).
   - `<abbr title="...">` — Abbreviation with expansion.

4. Structural containers (landmarks):
   - `<header>` — Introductory content (logo, nav, hero).
   - `<nav>` — Primary navigation links.
   - `<main>` — Dominant content of the `<body>` (unique per page).
   - `<section>` — Thematic grouping with a heading.
   - `<article>` — Self-contained composition (blog post, news item, product card).
   - `<aside>` — Tangentially related content (sidebar, pull quote, ads).
   - `<footer>` — Footer for nearest sectioning root (author, copyright, related links).
   - `<div>` — Generic container only when no semantic tag fits (styling hook).
5. Blockquotes and citations: `<blockquote cite="url">` for long quotes (block-level), `<q>` for short inline quotes (browser adds quotation marks).
6. Horizontal rule: `<hr>` — thematic break (paragraph-level), not a visual line.

KEY POINTS **Key Idea**

"Semantic HTML" means the tag name describes the content, not the look. Browsers, search engines (Google), and screen readers (for visually impaired users) rely on this.

WORKED EXAMPLE 6 **Marking Up a Blog Post on "Mobile Money in Cameroon"**

Statement: Write the HTML code for an `<article>` representing a blog post.

- Title: "The Rise of Mobile Money in Cameroon" (`h2` level).
- Author/Date byline: "By Ngwa Eric —Published 15 October 2024".
- Intro paragraph: "Mobile Money has transformed financial inclusion in Cameroon. Services like MTN MoMo and Orange Money allow users to send money, pay bills, and save without a bank account."
- Section "Key Statistics" (`h3`) with a paragraph: "According to a 2023 BEAC report, transaction volumes exceeded 15 trillion FCFA."
- A blockquote from the Minister of Posts and Telecommunications: "Digital finance is a pillar of our emergence strategy." (cite attribute: https://www.minpostel.gov.cm).
- Conclusion paragraph marked as an `<aside>` with a "Did you know?" style note: "Cameroon has one of the highest Mobile Money adoption rates in CEMAC."

SOLUTION

Detailed Solution:

We apply the semantic tags: `article`, `h2`, `h3`, `p`, `strong`, `em`, `blockquote`, `aside`. We use `<time>` for the date (machine-readable).

```
<article>
 <header>
 <h2>The Rise of Mobile Money in Cameroon</h2>
 <p>By Ngwa Eric -Published <time datetime="2024-10-15">15 October
2024</time></p>
 </header>
 <p>Mobile Money has transformed financial inclusion in Cameroon. Services
like <em>MTN MoMo</em> and <em>Orange Money</em> allow users to send money,
pay bills, and save without a bank account.</p>
 <section>
 <h3>Key Statistics</h3>
 <p>According to a 2023 BEAC report, transaction volumes exceeded <strong>15
trillion FCFA</strong>.</p>
 </section>
 <blockquote cite="https://www.minpostel.gov.cm">
 <p>Digital finance is a pillar of our emergence strategy.</p>
 </blockquote>
 <aside>
 <p>Did you know? Cameroon has one of the highest Mobile Money adoption
rates in CEMAC.</p>
 </aside>
</article>
```

Examiner's Comments:

- `<time datetime="...">` provides a machine-readable date (ISO 8601) while showing a human-friendly format. This is a distinction-level detail.
- `<em>` used for service names (convention for names of products/services). `<strong>` used for the critical statistic.
- `<blockquote>` includes the `cite` attribute (URL of source). The quote itself is wrapped in `<p>` inside the blockquote.
- The "Did you know?" box is tangentially related, so `<aside>` is correct, not a new `<section>`.

WARNING **Common Mistakes in Text Structure**

- Using `<h1>` multiple times on one page (confuses outline).
- Using `<br>` or empty `<p></p>` for spacing (use CSS `margin` or `padding`).
- Using `<b>` and `<i>` for importance/emphasis (they are presentational; use `<strong>`/`<em>`).
- Wrapping block elements (`<div>`, `<p>`) inside inline elements (`<a>`, `<span>`, `<strong>`) — invalid nesting.
- Forgetting `<p>` inside `<blockquote>` (required by HTML5).

7.3 Links, Images, Lists and Tables

METHOD **Creating Hyperlinks and Embedding Images Correctly**

Objective: Implement navigation and media using `<a>` and `<img>` with proper attributes for accessibility and UX.

1. Anchor `<a href="URL">`:
   - Absolute URL: `https://www.antric.cm` (external sites).
   - Relative URL: `about.html`, `../index.html`, `/contact.html` (internal navigation). Master relative paths for the exam.
   - Fragment/ID: `href="#contact"` (scroll to `id="contact"` on same page).
   - Email/Phone: `mailto:info@example.com`, `tel:+2376XXXXXXXX`.
   - Attributes: `target="_blank"` (new tab —add `rel="noopener noreferrer"` for security), `title="Tooltip text"`, `aria-label="..."` (accessible name when content is icon-only).
   - Current page: Add `aria-current="page"` to the active navigation link.
2. Image `<img>` (void element):
   - `src="path/to/image.jpg"` — Relative path from the HTML file.
   - `alt="Meaningful description"` — MANDATORY. Screen readers read this; shown if image fails. Empty `alt=""` only for purely decorative images (when `role="presentation"` or CSS background is better).
   - `width` & `height` — Reserve layout space (prevents Cumulative Layout Shift).
   - `loading="lazy"` — Defers loading off-screen images (performance).
   - `decoding="async"` — Non-blocking decode.

3. Responsive Images (Advanced): `<picture>` with `<source media="(max-width: 600px)" srcset="small.webp" type="image/webp">` and fallback `<img>`. Good to know for "integration activities".
4. Figure/Caption: Wrap `<img>` (or diagram, code snippet) in `<figure>` with `<figcaption>` for self-contained content with caption.

KEY POINTS **Exam Trap**

Forgetting `alt` attribute = accessibility fail = lost marks. Writing `alt="image"` or `alt="logo"` is insufficient; describe the function or content (e.g., `alt="MTN MoMo logo linking to homepage"`).

WORKED EXAMPLE 7 **Navigation Bar and Hero Image for a Douala E-commerce Site**

Statement: Code the `<header>` for "Douala Shop". It must contain:

1. A logo image `images/logo.png` (width 120px, height 40px) that links to `index.html`. Alt text: "Douala Shop Home".
2. A `<nav>` with an unordered list of links: Home (`index.html`), Products (`products.html`), Contact (`contact.html#form`).
3. A "Call Now" button linking to `tel:+237670000000`.

Below the header, code a `<main>` hero section with a background promotional image `images/promo.jpg` (width 1200, height 400) using `<img>` (not CSS background), with alt text "Special offer: 50% off all smartphones this weekend".

SOLUTION

Detailed Solution:
We combine `<a>` wrapping `<img>` for the logo, semantic `<nav>` `<ul>` for menu, and `tel:` protocol.

```
<header>
 <a href="index.html" aria-label="Douala Shop Home">
 <img src="images/logo.png" alt="" width="120" height="40">
 </a>
 <nav aria-label="Main navigation">
 <ul>
 <li><a href="index.html" aria-current="page">Home</a></li>
 <li><a href="products.html">Products</a></li>
 <li><a href="contact.html#form">Contact</a></li>
 </ul>
 </nav>
 <a href="tel:+237670000000" class="btn-call">Call Now</a>
</header>
<main>
 <section class="hero">
 <img src="images/promo.jpg" alt="Special offer: 50% off all smartphones
this weekend" width="1200" height="400" loading="lazy">
 </section>
</main>
```

Examiner's Comments:

- The logo is wrapped in `<a>`: clicking the image goes home. This is standard UX. The `<img>` has `alt=""` because the `<a>` provides `aria-label`.
- `width` and `height` on `<img>` are present (prevents layout jump).
- `loading="lazy"` on the hero image is debatable (hero is usually above fold), but acceptable as "good practice" in modern HTML. For exam, it shows awareness.
- The Contact link uses a fragment identifier `#form` targeting an element with `id="form"` on the contact page. Note the escaped `#` in the code table.
- `tel:+237670000000`: Cameroon country code is +237. On mobile, this triggers the dialer.
- Note the escaped % in alt text: `50%` (inside attribute values, % is safe, but & must be &).

METHOD **Building Lists and Accessible Data Tables**

Objective: Structure data using lists (`<ul>`, `<ol>`, `<dl>`) and tables (`<table>`) with proper semantics.

1. Lists:
   - `<ul>` — Unordered (bullets). Order doesn't matter.
   - `<ol>` — Ordered (numbers). Order matters (steps, ranking). Attributes: `start`, `reversed`, `type`.
   - `<dl>` — Description list (terms `<dt>` and definitions `<dd>`). Great for FAQs, glossaries, metadata.
   - Nesting: A `<li>` can contain another `<ul>` or `<ol>`. Indent correctly.
2. Tables — Structure:
   - `<table>` root.
   - `<caption>` — Title/description of table (immediately after `<table>`). Vital for accessibility.
   - `<thead>`, `<tbody>`, `<tfoot>` — Row groups.
   - `<tr>` (table row) → `<th>` (header cell) or `<td>` (data cell).
3. Table Header Association (Critical for GCE):
   - Use `scope="col"` on `<th>` in `<thead>`.
   - Use `scope="row"` on first `<th>` in `<tbody>` rows.
   - For complex tables, use `id` on `<th>` and `headers` attribute on `<td>`.
4. Merging Cells: `colspan="n"`, `rowspan="n"`.
5. Styling Hook: Add `class` to table for CSS (e.g., `class="data-table"`). Do not use `border`, `cellpadding`, `cellspacing`, `align`, `width` attributes (obsolete).

KEY POINTS **GCE Focus**

Exam questions often ask for a "well-structured table". Marks are awarded for: `<caption>`, `<thead>`/`<tbody>`, `scope` attributes, and correct nesting. Never use tables for page layout!

WORKED EXAMPLE 8 **Timetable for GCE ICT Practical Revision**

Statement: Create an HTML table showing a weekly revision timetable.

- Caption: "Upper Sixth ICT Revision Schedule —Week 1".
- Columns: Time, Monday, Tuesday, Wednesday, Thursday, Friday.
- Rows (Time slots): 08:00–10:00, 10:15–12:15, 12:15–13:15, 13:15–15:15.
- Data:
  - 08:00–10:00: Mon (HTML Structure), Tue (CSS Box Model), Wed (JS DOM), Thu (HTML Forms), Fri (Mock Practical).
  - 10:15–12:15: Mon (CSS Selectors), Tue (Flexbox/Grid), Wed (JS Events), Thu (Form Validation), Fri (Mock Practical).
  - 12:15–13:15: LUNCH BREAK (merged across all 5 days).
  - 13:15–15:15: Mon (Semantic Tags), Tue (Responsive), Wed (API Fetch), Thu (Project Work), Fri (Review).
- First column (Time) acts as row headers.

SOLUTION

Detailed Solution:
We use `<caption>`, `<thead>` with `scope="col"`, `<tbody>` with `scope="row"` on the first cell of each row. The lunch break uses `colspan="5"` (spanning the 5 day columns; the Time column remains separate).

```
<table class="schedule">
 <caption>Upper Sixth ICT Revision Schedule -Week 1</caption>
 <thead>
 <tr>
 <th scope="col">Time</th>
 <th scope="col">Monday</th>
 <th scope="col">Tuesday</th>
 <th scope="col">Wednesday</th>
 <th scope="col">Thursday</th>
 <th scope="col">Friday</th>
 </tr>
 </thead>
 <tbody>
 <tr>
 <th scope="row">08:00-10:00</th>
 <td>HTML Structure</td>
 <td>CSS Box Model</td>
 <td>JS DOM</td>
 <td>HTML Forms</td>
 <td>Mock Practical</td>
 </tr>
 <tr>
 <th scope="row">10:15-12:15</th>
 <td>CSS Selectors</td>
 <td>Flexbox/Grid</td>
 <td>JS Events</td>
 <td>Form Validation</td>
 <td>Mock Practical</td>
 </tr>
 <tr>
 <th scope="row">12:15-13:15</th>
 <td colspan="5">LUNCH BREAK</td>
 </tr>
 <tr>
 <th scope="row">13:15-15:15</th>
 <td>Semantic Tags</td>
 <td>Responsive Design</td>
 <td>API Fetch</td>
 <td>Project Work</td>
 <td>Review</td>
```

Examiner's Comments:

- `scope="col"` on header row cells; `scope="row"` on the first cell (`<th>`) of each body row. This explicitly links headers to data cells for screen readers.
- The Lunch Break row: The first cell is a `<th scope="row">` (Time label), then a single `<td colspan="5">` spanning the 5 day columns. Total columns = 6.
- No deprecated attributes (`border`, `cellspacing`, `align`). Styling is left to CSS via `class="schedule"`.
- `<caption>` is the first child of `<table>`.

EXPERIMENT **Building a Description List for ICT Glossary**

Task: Create a `<dl>` for key ICT terms: "HTML", "CSS", "JavaScript", "API". Each term (`<dt>`) followed by its definition (`<dd>`). Solution:

```
<dl class="glossary">
 <dt>HTML</dt>
 <dd>HyperText Markup Language -structures web content.</dd>
 <dt>CSS</dt>
 <dd>Cascading Style Sheets -styles presentation.</dd>
 <dt>JavaScript</dt>
 <dd>Programming language for dynamic web behaviour.</dd>
 <dt>API</dt>
 <dd>Application Programming Interface -allows software components to
communicate.</dd>
</dl>
```

Note: A `<dt>` can have multiple `<dd>` (multiple definitions) and vice versa.

7.4 Forms, Validation and Integration Project

METHOD **Designing Accessible HTML Forms with Client-Side Validation**

Objective: Create forms that are usable, accessible, and leverage browser built-in validation (HTML5 Constraint Validation API).

1. Form Wrapper: `<form action="/submit-url" method="POST">`. `method="POST"` for data changes (registration, payment); `GET` for search/filter (idempotent).

2. Labeling (Non-negotiable): Every `<input>`, `<select>`, `<textarea>` MUST have an associated `<label>`.
   - Explicit (preferred): `<label for="email">Email</label> <input id="email" ...>`.
   - Implicit: `<label>Email <input ...></label>` (allowed but less flexible for CSS).
3. Input Types (Use the right one!):
   - `text`, `email`, `tel`, `url`, `password`, `number`, `date`, `datetime-local`, `month`, `week`, `time`, `checkbox`, `radio`, `file`, `hidden`, `submit`, `reset`, `button`, `color`, `range`, `search`.
   - `email`, `url`, `tel`, `number`, `date` trigger native validation and appropriate mobile keyboards (crucial for Cameroon's mobile-first users).
4. Validation Attributes:
   - `required` — Field cannot be empty.
   - `minlength`, `maxlength`, `min`, `max`, `step` (for number/date).
   - `pattern="regex"` — Custom regex (e.g., Cameroon phone: `pattern="[236789][0-9]8"` for 9 digits starting with 2,3,6,7,8,9).
   - `title` — Tooltip hint for pattern (shown on validation error in some browsers).
   - `accept` — File types for `<input type="file">` (e.g., `accept=".pdf,image/*"`).
   - `multiple` — Allow multiple files/emails.
5. Grouping: `<fieldset>` with `<legend>` for related controls (radio groups, checkboxes, address block). `<fieldset disabled>` disables all descendants.
6. Autocomplete: `autocomplete="name"`, `"email"`, `"tel"`, `"current-password"`, `"new-password"`, `"off"` — helps password managers and browsers.
7. Feedback: `<output>` for calculated results. CSS pseudo-classes `:valid`, `:invalid`, `:required`, `:optional`, `:user-invalid` (modern), `:in-range`, `:out-of-range` style states.
8. Submit: `<button type="submit">Register</button>` (preferred over `<input type="submit">` for styling flexibility and HTML content). `<button type="reset">` resets form; `<button type="button">` for JS actions.
9. Constraint Validation API (JS): `input.checkValidity()`, `input.reportValidity()`, `input.setCustomValidity("msg")`, `form.reportValidity()`, `form.checkValidity()`. The `novalidate` attribute on `<form>` disables browser UI bubbles (for custom JS validation).

KEY POINTS **Security Note**

Client-side validation (HTML5/JS) is for UX only. It can be bypassed (disable JS, curl, Postman, dev tools). Server-side validation is mandatory. GCE questions may ask "Why is server-side validation still needed?".

WORKED EXAMPLE 9 **Registration Form for "Cameroon Digital Skills Portal"**

Statement: Write the HTML for a registration form with the following fields. Use appropriate HTML5 types and validation attributes. No JavaScript.

1. Full Name (text, required, max 100 chars).
2. Email (email type, required).
3. Phone Number (tel type, required, pattern for Cameroon format: 9 digits starting with 6 or 7, placeholder example "6XX XXXX XX").
4. Date of Birth (date type, required, user must be at least 16 years old —assume max date is today minus 16 years, use `max` attribute with a fixed date like `2008-12-31` for the exam context).
5. Gender (radio buttons: Male, Female, Other —required, grouped in fieldset).
6. Skills (checkboxes: HTML, CSS, JavaScript, Python —at least one required note: HTML5 cannot enforce "at least one checkbox" natively without JS, so just mark individual as required or leave optional; here we make them optional).
7. CV Upload (file type, accept PDF only, required).
8. Submit button "Create Account".

SOLUTION

Detailed Solution:
We structure with `<form>`, explicit labels (`for`/`id`), `<fieldset>` for radio group, and HTML5 validation attributes. Note: "At least one checkbox" requires JS, so we omit `required` on checkboxes.

```
<form action="/register" method="POST" class="reg-form">
 <div class="field">
 <label for="fullname">Full Name</label>
 <input type="text" id="fullname" name="fullname" required maxlength="100"
autocomplete="name">
 </div>
 <div class="field">
 <label for="email">Email Address</label>
 <input type="email" id="email" name="email" required autocomplete="email">
 </div>
 <div class="field">
 <label for="phone">Phone Number (Cameroon)</label>
 <input type="tel" id="phone" name="phone" required
 pattern="[67][0-9]8"
 placeholder="6XX XXXX XX or 7XX XXXX XX"
 title="9 digits starting with 6 or 7 (e.g. 670000000)">
 </div>
 <div class="field">
 <label for="dob">Date of Birth (Age ≥ 16)</label>
 <input type="date" id="dob" name="dob" required max="2008-12-31">
 </div>
 <fieldset class="field">
 <legend>Gender</legend>
 <div><label><input type="radio" name="gender" value="male" required>
Male</label></div>
 <div><label><input type="radio" name="gender" value="female">
Female</label></div>
 <div><label><input type="radio" name="gender" value="other">
Other</label></div>
 </fieldset>
 <fieldset class="field">
 <legend>Skills (Optional)</legend>
 <div><label><input type="checkbox" name="skills" value="html">
HTML</label></div>
 <div><label><input type="checkbox" name="skills" value="css">
CSS</label></div>
 <div><label><input type="checkbox" name="skills" value="js">
JavaScript</label></div>
 <div><label><input type="checkbox" name="skills" value="python">
Python</label></div>
 </fieldset>
 <div class="field">
```

Examiner's Comments:

- For pure HTML5 validation demo, omit `novalidate` to let browser show bubbles. (I included `novalidate` in the method as "best practice for real dev with custom JS", but for GCE "HTML5 validation" questions, remove it.)
- `autocomplete` attributes help password managers/browsers fill forms correctly (accessibility/UX marks).
- Phone pattern: `[67][0-9]8` matches 6 or 7 followed by 8 digits (total 9). Cameroonian numbers are 9 digits.
- `max="2008-12-31"` on date input enforces the 16-year rule client-side.
- Radio group: `required` on one input in the group (the first) is sufficient for the browser to enforce selection.
- File input: `accept=".pdf,application/pdf"` filters file picker. Server must still check MIME type.
- `<button type="submit">` allows HTML content inside (icons, spans).

METHOD **Integration Project: Structuring a Complete Multi-Page Site**

Objective: Assemble all skills into a coherent mini-site structure (Integration Activity — Lesson 45). Focus on file organization, shared layout, and navigation consistency.

1. File Structure (Root Folder):

```
project-root/
|| index.html
|| about.html
|| services.html
|| contact.html
|| css/
|| || style.css
|| images/
|| || logo.png
|| || hero.jpg
|| || team.jpg
|| js/
|| || main.js
```

2. Shared Layout Template: Create a "master" HTML structure (header, nav, footer) and

duplicate for each page, changing only `<main>` content and `<title>`.

- In `<nav>`, add `aria-current="page"` to the link of the current page (accessibility).
- Use `<a href="index.html">Home</a>` etc. Relative paths work because all HTML files are in root.

3. Consistent IDs for Anchors: `<main id="main-content">` allows "Skip to main content" link at top of body (accessibility).
4. Meta Tags per Page: Unique `<title>`, `<meta name="description">` for SEO.
5. Testing Checklist (Exam Simulation):
   (a) Validate all HTML files (W3C Validator).
   (b) Test all internal links (click every nav link).
   (c) Test form submission (check `action` URL, `method`).
   (d) Test responsive: resize browser window (viewport meta).
   (e) Check images load (paths correct, `alt` text present).
   (f) Check console for errors (F12).

KEY POINTS **Exam Strategy**

In a 3-hour practical, spend 30 mins setting up the folder structure and the `index.html` skeleton perfectly. Copy it to `about.html`, `services.html`, `contact.html` before filling content. This guarantees consistent navigation and structure.

WORKED EXAMPLE 10 **Mini-Site "Yaoundé Events" —3 Pages Structure**

Statement: You are building a 3-page site for "Yaoundé Events" (Home, Events, Contact).

1. Draw the folder tree (text-based).
2. Write the complete HTML for `index.html` (Homepage).
   - Title: "Yaoundé Events —Your Guide to Capital Fun".
   - Header: Logo (`images/logo.svg`, alt "Yaoundé Events Home"), Nav (Home, Events, Contact).
   - Main: Hero section with `<h1>` "Discover Yaoundé", intro paragraph, "View Events" button linking to `events.html`.

- Footer: Copyright, Social links (Facebook, Twitter/X icons as images with alt text), Link to `mentions-legales.html` (not created).

3. Write only the `<main>` content for `events.html` (assume same header/footer). It should contain a `<section>` with an `<article>` for one event: "Ngondo Festival", Date, Location (Douala —but listed here), Description, "Register" button linking to `contact.html#registration`.

SOLUTION

Detailed Solution:

1. Folder Tree:

```
yaounde-events/
|| index.html
|| events.html
|| contact.html
|| css/
|| || style.css
|| images/
|| || logo.svg
|| || hero.jpg
|| || ngondo.jpg
|| js/
|| || main.js
```

2. `index.html` (Complete):

```
<!DOCTYPE html>
<html lang="en">
<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <meta name="description" content="Discover the best events in Yaoundé:
concerts, festivals, conferences and nightlife.">
 <title>Yaoundé Events -Your Guide to Capital Fun</title>
 <link rel="stylesheet" href="css/style.css">
</head>
<body>
 <a href="#main-content" class="skip-link">Skip to main content</a>
 <header class="site-header">
 <a href="index.html" class="logo" aria-label="Yaoundé Events Home">
 <img src="images/logo.svg" alt="" width="120" height="40">
 </a>
 <nav class="main-nav" aria-label="Main navigation">
 <ul>
 <li><a href="index.html" aria-current="page">Home</a></li>
 <li><a href="events.html">Events</a></li>
 <li><a href="contact.html">Contact</a></li>
 </ul>
 </nav>
 </header>
 <main id="main-content">
 <section class="hero">
 <h1>Discover Yaoundé</h1>
 <p>From the vibrant streets of Mvog-Ada to the cultural centres of Bastos,
find your next adventure.</p>
 <a href="events.html" class="btn btn-primary">View Events</a>
 </section>
 <section class="featured">
 <h2>Popular Categories</h2>
 <ul class="cat-list">
 <li><a href="events.html?cat=music">Music & Concerts</a></li>
 <li><a href="events.html?cat=tech">Tech & Innovation</a></li>
 <li><a href="events.html?cat=culture">Culture & Arts</a></li>
 </ul>
 </section>
 </main>
 <footer class="site-footer">
```

3. `events.html` —`<main>` Content Only:

```
<main id="main-content">
 <header class="page-header">
 <h1>Upcoming Events</h1>
 <p>Filter by category or date to find your perfect match.</p>
 </header>
 <section class="event-list">
 <article class="event-card">
 <img src="images/ngondo.jpg" alt="Ngondo Festival traditional boat race on
Wouri River" width="600" height="400" loading="lazy">
 <div class="event-details">
 <time datetime="2025-12-01">1st December 2025</time>
 <h2>Ngondo Festival</h2>
 <p class="location">Douala, Wouri River Banks</p>
 <p>The annual celebration of the Sawa people. Traditional boat races,
cultural dances, and the sacred immersion ceremony.</p>
 <a href="contact.html#registration" class="btn">Register Interest</a>
 </div>
 </article>
 <!-- More articles would go here -->
 </section>
</main>
```

Examiner's Comments:

- Skip Link: `<a href="#main-content" class="skip-link">` is the first focusable element. Hidden visually via CSS until focused. Major accessibility mark.
- `aria-current="page"` on active nav link tells screen readers "You are here".
- Logo image has `alt=""` (empty) because the `<a>` has `aria-label` providing the accessible name. This avoids duplication.
- `rel="noopener noreferrer"` on external social links (security).
- `events.html` main uses `<time datetime="...">` for machine-readable date.
- Query string in category links: `events.html?cat=music` —prepares for dynamic filtering (JS or server-side).
- Consistent `id="main-content"` on both pages allows the skip link to work everywhere.
- Note escaped & in "Music & Concerts" and "Tech & Innovation" inside attribute values.

METHOD **Debugging, Validation and Deployment Basics**

Objective: Ensure the site works, is standards-compliant, and can be published.

1. Validation Workflow:
   (a) Save file (`Ctrl+S`).
   (b) Open in browser (`F5` / Live Server).
   (c) Open DevTools (`F12`) → Console (red errors), Elements (inspect DOM).
   (d) Copy HTML source → https://validator.w3.org/ → "Validate by Direct Input". Fix all Errors. Warnings are optional but recommended.
2. Common GCE Practical Errors to Hunt:
   - Unclosed tags (`<p>Text</p>` missing).
   - Wrong nesting (`<a><div></a></div>` — block inside inline).
   - Duplicate `id` values (must be unique per page).
   - Missing `alt` on images.
   - `<label for="x">` but no `id="x"` on input.
   - Obsolete attributes: `align`, `border`, `width` on `<table>`, `<body bgcolor>`.
   - Character encoding issues: & written as & (must be & in attributes/text), < as <.
   - Missing `<meta charset="UTF-8">` or `<title>`.
   - `<input>` without `name` attribute (data not submitted).
3. Cross-Browser Testing: Test in Chrome/Edge (Chromium) AND Firefox (Gecko). Mobile view (Device Toolbar in DevTools).
4. Deployment (Conceptual):
   - Static hosting: GitHub Pages, Netlify, Vercel (free, HTTPS automatic).
   - Traditional: FTP/SFTP to `public_html` or `www` on a LAMP server (CAMTEL, MTN Business, or private hosting).
   - Domain: `.cm` (managed by ANTIC), `.com`, `.org`. DNS: A record (IP), CNAME (alias).
5. Performance Basics: Minify CSS/JS (remove whitespace), compress images (WebP, AVIF), enable Gzip/Brotli on server, leverage browser caching (HTTP headers).

KEY POINTS **Exam Tip**

If the practical exam asks "Test your website", explicitly write in your report: "Validated HTML5 via W3C Validator —0 Errors. Tested on Chrome and Firefox. Responsive layout verified at 320px, 768px, 1024px. All links functional. Form submits data to `action` URL."

WORKED EXAMPLE 11 **Debugging a Broken "Contact" Page**

Statement: A student wrote the following `contact.html` code. It renders poorly in the browser: the form fields are not aligned, the "Send" button does nothing, and the validator reports errors. Identify 5 distinct errors (structural, semantic, or attribute-related) and provide the corrected code block for the `<main>` section only.
Student's Code (Main Section Only):

```
<main>
 <h1>Contact Us</h1>
 <form action="send.php" method="GET">
 <div>
 <label>Name:</label>
 <input type="text" name="name" required>
 </div>
 <div>
 <label>Email:</label>
 <input type="text" name="email" required>
 </div>
 <div>
 <label>Message:</label>
<textarea name="msg" rows="5" cols="30"></textarea>
 </div>
 <div>
 <input type="submit" value="Send">
 <input type="reset" value="Clear">
 </div>
 </form>
 <div id="footer">
 <p>Copyright 2025</p>
 </div>
</main>
```

SOLUTION

Detailed Solution:
Error Analysis (5+ found):

1. Form Method GET: Sending user data (name, email, message) via `method="GET"` exposes data in URL history/logs, has length limits, and is semantically wrong for "sending" data. Fix: `method="POST"`.
2. Missing `for`/`id` Association: Labels wrap text but lack `for` attribute matching input `id`. Inputs lack `id` entirely. Clicking label won't focus input. Fix: Add `id` to inputs, `for="id"` to labels.
3. Wrong Input Type for Email: `type="text"` for email disables native validation and mo-

bile keyboard optimization. Fix: `type="email"`.

4. Textarea Accessibility/Structure: `<textarea>` has no associated label (same issue as above). Also, `rows/cols` attributes are presentational; CSS should handle sizing, but they are valid attributes. Main issue: missing `id/for`.
5. Reset Button UX Hazard: `<input type="reset">` clears the form instantly on accidental click. Rarely used in modern UX. Fix: Remove or replace with `<button type="reset">` if absolutely required, but better omitted.
6. Footer inside Main: `<div id="footer">` is inside `<main>`. Footer is site-wide, not main content. Should be `<footer>` sibling of `<main>`. Also `div` instead of semantic `<footer>`.
7. Missing `name` on Submit: Not strictly an error, but good practice to give submit button a `name` if backend checks which button was pressed.
8. Character Encoding in Copyright: "Copyright 2025" should ideally use `©` or ©.

Corrected `<main>` Section (and footer moved out):

```
<main>
 <h1>Contact Us</h1>
 <form action="send.php" method="POST">
 <div class="field">
 <label for="name">Name:</label>
 <input type="text" id="name" name="name" required autocomplete="name">
 </div>
 <div class="field">
 <label for="email">Email:</label>
 <input type="email" id="email" name="email" required autocomplete="email">
 </div>
 <div class="field">
 <label for="message">Message:</label>
 <textarea id="message" name="message" rows="5" required></textarea>
 </div>
 <div class="field-actions">
 <button type="submit" name="submit-contact">Send Message</button>
 <!-- Reset button removed for better UX -->
 </div>
 </form>
</main>
<footer>
 <p>&copy; 2025 Yaoundé Events. All rights reserved.</p>
</footer>
```

Examiner's Comments:

- The correction moves `<footer>` outside `<main>` (structural correction).
- Uses `<button type="submit">` instead of `<input type="submit">` (modern, styleable).
- Adds `autocomplete` hints.
- Removed `cols` from textarea (CSS handles width).
- Fixed `method="POST"`.
- Added `name` attribute to textarea (`message` instead of `msg` for clarity).

EXERCISE 5 **Practice Questions for GCE Preparation**

1. Write the complete HTML5 skeleton for a page titled "Bamenda Cultural Festival". Include viewport meta, UTF-8 charset, link to `festival.css`, and semantic landmarks (`header`, `nav`, `main`, `footer`) with empty content.
2. Create a semantic `<article>` for a news item: Headline "MTN Cameroon Launches 5G Trials in Yaoundé", date 2025-03-15, author "Tech Reporter", two paragraphs of content, a `<blockquote>` from the CEO with cite attribute, and an `<aside>` with a "Fact: 5G latency < 10ms" note.
3. Code a navigation bar with logo (image `logo.png`, 100×30, link to home), three nav links (Home, Services, Contact), and a "WhatsApp Chat" link using `https://wa.me/2376XXXXXXXX`. Ensure accessibility attributes.
4. Build a table "GCE ICT Practical Marks" with caption, columns: Student ID, Name, HTML (20), CSS (20), JS (20), Total (60). Three rows of data. First column and header row must use `scope` attributes. Add a footer row with averages (use `<tfoot>`).
5. Design a login form: Email (required, email type), Password (required, password type, minlength 8), "Remember me" checkbox, Submit button. Use explicit labels, fieldset for the checkbox if alone, and HTML5 validation only.
6. Integration: Sketch the folder structure for a 5-page site "Limbe Tourism" (Home, Attractions, Hotels, Gallery, Contact) with css, images, js folders. Write the `<nav>` HTML for the Home page (with `aria-current`) and the `<main>` for Attractions page containing two `<article>` cards (image, `<h3>`, short description, "Read more" link).

SOLUTION **Exercise 1**

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Bamenda Cultural Festival</title>
<link rel="stylesheet" href="festival.css">
</head>
<body>
<header></header>
<nav></nav>
<main></main>
<footer></footer>
</body>
```

```
</html>
```

SOLUTION **Exercise 2**

```
<article>
<header>
<h2>MTN Cameroon Launches 5G Trials in Yaoundé</h2>
<p>By Tech Reporter -<time datetime="2025-03-15">15 March 2025</time></p>
</header>
<p>MTN Cameroon has announced the start of 5G network trials in the capital city
Yaoundé, marking a major milestone for the country's digital transformation.</p>
<p>The trials will cover key business districts and university campuses,
offering speeds up to 1 Gbps to selected users.</p>
<blockquote cite="https://www.mtn.cm">
<p>5G will unlock new opportunities for innovation, education, and healthcare
across Cameroon.</p>
</blockquote>
<aside>
<p>Fact: 5G latency < 10ms enables real-time applications like remote surgery
and autonomous vehicles.</p>
</aside>
</article>
```

SOLUTION **Exercise 3**

```
<header>
<a href="index.html" aria-label="Home">
<img src="logo.png" alt="" width="100" height="30">
</a>
<nav aria-label="Main navigation">
<ul>
<li><a href="index.html" aria-current="page">Home</a></li>
<li><a href="services.html">Services</a></li>
<li><a href="contact.html">Contact</a></li>
</ul>
</nav>
<a href="https://wa.me/2376XXXXXXXX" target="_blank" rel="noopener noreferrer"
aria-label="Chat on WhatsApp">WhatsApp Chat</a>
</header>
```

SOLUTION **Exercise 4**

```
<table class="marks">
<caption>GCE ICT Practical Marks</caption>
<thead>
<tr>
<th scope="col">Student ID</th>
<th scope="col">Name</th>
<th scope="col">HTML (20)</th>
<th scope="col">CSS (20)</th>
<th scope="col">JS (20)</th>
<th scope="col">Total (60)</th>
</tr>
</thead>
<tbody>
<tr>
<th scope="row">U6CS001</th>
<td>Ngwa Eric</td>
<td>18</td>
<td>19</td>
<td>17</td>
<td>54</td>
</tr>
<tr>
<th scope="row">U6CS002</th>
<td>Fon Grace</td>
<td>15</td>
<td>16</td>
<td>18</td>
<td>49</td>
</tr>
<tr>
<th scope="row">U6CS003</th>
<td>Tchinda Paul</td>
<td>20</td>
<td>18</td>
<td>19</td>
<td>57</td>
</tr>
</tbody>
<tfoot>
<tr>
<th scope="row">Average</th>
<td></td>
```

```
<td>17.7</td>
<td>17.7</td>
<td>18.0</td>
<td>53.3</td>
</tr>
</tfoot>
</table>
```

SOLUTION **Exercise 5**

```
<form action="/login" method="POST">
<div class="field">
<label for="email">Email</label>
<input type="email" id="email" name="email" required autocomplete="email">
</div>
<div class="field">
<label for="pwd">Password</label>
<input type="password" id="pwd" name="password" required minlength="8"
autocomplete="current-password">
</div>
<div class="field">
<label for="remember">
<input type="checkbox" id="remember" name="remember" value="1"> Remember me
</label>
</div>
<button type="submit">Sign In</button>
</form>
```

SOLUTION **Exercise 6**

Folder Structure:

```
limbe-tourism/
|| index.html
|| attractions.html
|| hotels.html
|| gallery.html
|| contact.html
|| css/
|| || style.css
|| images/
|| || logo.png
|| || hero.jpg
|| || limbe-botanic.jpg
|| || mount-cameroon.jpg
|| js/
|| || main.js
```

Home page `<nav>`: `<nav aria-label="Main navigation">`

```
<ul>
<li><a href="index.html" aria-current="page">Home</a></li>
<li><a href="attractions.html">Attractions</a></li>
<li><a href="hotels.html">Hotels</a></li>
<li><a href="gallery.html">Gallery</a></li>
<li><a href="contact.html">Contact</a></li>
</ul>
</nav>
```

Attractions page `<main>`: `<main id="main-content">`

```
<header><h1>Top Attractions in Limbe</h1></header>
<section class="attractions-grid">
<article class="card">
<img src="images/limbe-botanic.jpg" alt="Limbe Botanic Garden entrance with
tropical trees" width="400" height="300" loading="lazy">
<div class="card-body">
<h3>Limbe Botanic Garden</h3>
<p>One of Africa's oldest botanical gardens, home to rare medicinal plants and a
quiet canopy walk.</p>
<a href="attractions.html#botanic" class="btn">Read more</a>
</div>
</article>
<article class="card">
<img src="images/mount-cameroon.jpg" alt="Mount Cameroon summit at sunrise"
width="400" height="300" loading="lazy">
```

```
<div class="card-body">
<h3>Mount Cameroon</h3>
<p>West Africa's highest peak (4,095m). Guided hikes from Buea offer
breathtaking views and volcanic landscapes.</p>
<a href="attractions.html#mount-cameroon" class="btn">Read more</a>
</div>
</article>
</section>
</main>
```

8 EXERCISES

8.1 I check my knowledge

EXERCISE 6 **MCQ: tags and attributes**

For each item, choose the single correct answer.

1. The root element of every HTML document is:
   (a) `<head>`
   (b) `<html>`
   (c) `<body>`
   (d) `<title>`
2. Which tag produces the largest heading?
   (a) `<h6>`
   (b) `<header>`
   (c) `<h1>`
   (d) `<head>`
3. The attribute that gives the address of a hyperlink is:
   (a) `src`
   (b) `link`
   (c) `href`
   (d) `url`
4. Which of the following is an empty element (it has no closing tag)?
   (a) `<p>`
   (b) `<img>`
   (c) `<li>`
   (d) `<a>`
5. To create a single-line text field in a form, one uses:
   (a) `<input type="text">`
   (b) `<textfield>`

(c) `<input type="field">`

(d) `<text>`

EXERCISE 7 **True or false**

Say whether each statement is true or false, and justify your answer in one sentence.

1. The content of the `<title>` element is displayed in the main window of the browser.
2. The `alt` attribute of an image is optional and can always be omitted without consequence.
3. An HTML file can be saved with the extension `.html` or `.htm`.
4. A relative URL gives the position of a file with respect to the current document.

EXERCISE 8 **Definitions**

Define each of the following terms in one or two sentences, giving an example where possible:

1. Hyperlink;
2. Empty element;
3. Attribute;
4. Absolute URL.

EXERCISE 9 **Matching tags to roles**

Copy and complete the following table by giving the role of each tag and one example of use.

Tag	Role	Example of use
`<ol>`		
`<td>`		
` `		
`<select>`		

8.2 I practise

EXERCISE 10 My first page

Write the complete HTML code of a page entitled My School containing: a level-1 heading with the name of your school, one paragraph presenting the school, and a level-2 heading Location followed by a short paragraph. The document must contain all the compulsory structural elements (`<!DOCTYPE>`, `<html>`, `<head>`, `<title>`, `<body>`).

EXERCISE 11 Connecting files

A web site is organised as follows: the file `index.html` is in the main folder; this folder also contains a subfolder `pages` with the file `about.html`, and a subfolder `images` with the file `logo.png`.

1. Write the `href` value of a link placed in `index.html` pointing to `about.html`.
2. Write the complete tag that displays `logo.png` inside `index.html`, with a suitable `alt` text.
3. Write the `href` value of a link placed in `about.html` returning to `index.html`.
4. Write the `src` value displaying `logo.png` from inside `about.html`.

EXERCISE 12 A marks table

Write the HTML code of a table showing the marks of three students, with three columns (Name, Subject, Mark), a header row, a caption Sequence 1 marks, and borders. In the last row, the cell Class average must span the first two columns.

EXERCISE 13 Canteen registration form

Write the HTML code of a form entitled Canteen registration containing: a text field for the student's name, two radio buttons (Day student / Boarder) of which only one can be selected, a drop-down list offering the classes Lower Sixth, Upper Sixth Science and Upper Sixth Arts, and a submit button labelled Register. Then explain in two sentences the role of the `name` attribute of the form controls.

8.3 I prepare for the GCE

EXERCISE 14 **Structured question: reproducing a page (20 marks)**

A teacher opens a web page in a browser and sees the following rendered result: the browser tab shows the title GCE Revision; the page displays a large heading Welcome to GCE Revision, a first paragraph This site helps Upper Sixth students prepare for the examination., a second paragraph in which the word free appears in bold, a bulleted list with the three items Mathematics, Physics and ICT, and finally an image stored in the file `study.png`.

1. Write the complete HTML code that produces exactly this page, including all structural elements. (12 marks)
2. State the purpose of the `alt` attribute of the image and give a suitable value for it here. (3 marks)
3. Explain the difference between the `<title>` element and the `<h1>` element as they appear in this page. (3 marks)
4. The teacher wants the word ICT in the list to open the page `ict.html` (in the same folder) when clicked. Give the modified line of code. (2 marks)

EXERCISE 15 **Error hunting (18 marks)**

The following HTML document, saved by a student as `page.txt`, contains exactly six mistakes. Identify each mistake, give its line number, and write the corrected line.

Line	Code
1	`<html>`
2	`<head>`
3	`<title>My page<title>`
4	`</head>`
5	`<body>`
6	`<h1>Our College</h1>`
7	`<p>Welcome to our site`
8	`<ul>`
9	`<li>Students</li>`
10	`</ul>`
11	`<li>Teachers</li>`
12	`<img src=photo.png>`
13	`<a href="contact.html">Contact<a>`
14	`</body>`
15	`</html>`

1. Identify and correct the six mistakes in the code above. (12 marks)
2. Explain why the file name `page.txt` prevents the document from being displayed as a web page, and state the correct extension. (3 marks)
3. Give two reasons why the `alt` attribute should be added to the image. (3 marks)

EXERCISE 16 **Essay question: the school web site (20 marks)**

The principal of a Government High School in Bamenda wants to publish a simple web site presenting the school, its classes and an online enrolment form. You are consulted as the ICT specialist.

1. Describe the structure of a well-formed HTML document, naming each structural element and its role. (6 marks)
2. Explain, with one example each, the difference between an absolute URL and a relative URL, and state when each is preferable. (6 marks)
3. The enrolment form must collect the candidate's name, class and sex. For each piece of information, propose the most appropriate form control and justify your choice. (4 marks)

4. Discuss two good practices that make the site usable by visitors with a slow connection or a visual disability. (4 marks)

9 SOLUTIONS TO THE EXERCISES

9.1 Solutions — I check my knowledge

SOLUTION **Exercise 1 — MCQ: tags and attributes**

1. Answer (b). `<html>` is the **root element** of every HTML document: it encloses the two structural blocks `<head>` (information about the page) and `<body>` (visible content). `<head>` and `<body>` are children of `<html>`, and `<title>` is a child of `<head>`.
2. Answer (c). HTML provides six heading levels, from `<h1>` (largest) down to `<h6>` (smallest). `<header>` is not a heading, and `<head>` is a structural element that is never displayed.
3. Answer (c). The attribute `href` (hypertext reference) carries the address of the destination: `<a href="page.html">`. `src` is used for embedded resources such as images; `link` and `url` are not attributes of `<a>`.
4. Answer (b). `<img>` is an **empty element**: it cannot contain content and has no closing tag. `<p>`, `<li>` and `<a>` all enclose content and therefore require a closing tag.
5. Answer (a). `<input type="text">` creates a single-line text field. The tags `<textfield>` and `<text>` and the type `"field"` do not exist in HTML.

SOLUTION **Exercise 2 — True or false**

1. False. The content of `<title>` appears in the browser tab (or title bar) and in search-engine results; only the content of `<body>` is displayed in the main window.
2. False. The `alt` text is essential for accessibility: it is read aloud by screen readers for visually impaired users and is shown in place of the image when the file cannot be loaded.
3. True. Browsers and web servers recognise both `.html` and `.htm` as HTML documents; `.html` is the recommended choice.
4. True. A relative URL such as `pages/about.html` is interpreted starting from the folder that contains the current document, which is precisely what makes it “relative”.

SOLUTION **Exercise 3 — Definitions**

1. Hyperlink: a clickable element of a web page that leads the user to another page, another web site, or another position within the same page. It is created with the anchor element, e.g. `<a href="about.html">About us</a>`.
2. Empty element: an element that cannot contain any content and therefore has no closing tag; examples are `<img>`, `<br>` and `<input>`.
3. Attribute: additional information written inside an opening tag in the form `name="value"`, which specifies or modifies the behaviour of the element; for example `src` in `<img src="logo.png">`.
4. Absolute URL: a complete address giving the protocol, the server name and the full path of a resource, for example `https://www.minesec.gov.cm/index.html`.

SOLUTION **Exercise 4 — Matching tags to roles**

Tag	Role	Example of use
`<ol>`	Ordered (numbered) list; items appear as 1, 2, 3, …	Listing the steps of a recipe or of an experiment, where order matters
`<td>`	Ordinary data cell of a table, placed inside a row `<tr>`	One student's mark in a marks table
` `	Line break inside a paragraph; it is an empty element	Writing a postal address on several lines
`<select>`	Drop-down list of options in a form, filled with `<option>` elements	Choosing one's class in a registration form

9.2 Solutions — I practise

SOLUTION **Exercise 5 — My first page**

A model answer (any real school name is acceptable):

HTML code

```
<!DOCTYPE html>
<html>
<head>
<title>My School</title>
</head>
<body>
<h1>Government High School Bamenda</h1>
<p>Our school offers general education from Form One to Upper Sixth.</p>
<h2>Location</h2>
<p>The school is situated in Bamenda, North West Region of Cameroon.</p>
</body>
</html>
```

Marking points: the document starts with `<!DOCTYPE html>`; `<title>` sits inside `<head>`; all visible content (headings and paragraphs) sits inside `<body>`; every opened tag is closed in the reverse order of opening.

SOLUTION **Exercise 6 — Connecting files**

1. `href="pages/about.html"` — from `index.html` we go down into the subfolder `pages`.
2. `<img src="images/logo.png" alt="School logo">`
3. `href="../index.html"` — the notation `../` means “go up one folder”, from `pages` back to the main folder.
4. `src="../images/logo.png"` — first go up to the main folder (`../`), then down into the `images` subfolder.

WARNING **Common trap**

Writing `href="/pages/about.html"` or using backslashes `\` copied from Windows file paths. In URLs, folders are always separated by a forward slash `/`, and a leading `/` refers to the server root, not to the current folder.

SOLUTION **Exercise 7 — A marks table**

HTML code

```
<table border="1">
<caption>Sequence 1 marks</caption>
<tr><th>Name</th><th>Subject</th><th>Mark</th></tr>
<tr><td>Amina</td><td>ICT</td><td>16</td></tr>
<tr><td>Bello</td><td>ICT</td><td>12</td></tr>
<tr><td>Clara</td><td>ICT</td><td>14</td></tr>
<tr><td colspan="2">Class average</td><td>14</td></tr>
</table>
```

Explanations:

- `<th>` cells form the header row (displayed in bold and centred by default);
- the attribute `colspan="2"` makes the cell Class average occupy the width of the first two columns, so the last row contains only two cells instead of three;
- the class average is indeed (16 + 12 + 14)/3 = 14.

SOLUTION **Exercise 8 — Canteen registration form**

HTML code

```
<h1>Canteen registration</h1>
<form action="register.php" method="post">
<p>Name: <input type="text" name="studentname"></p>
<p><input type="radio" name="status" value="day"> Day student
<input type="radio" name="status" value="boarder"> Boarder</p>
<p>Class: <select name="class">
<option>Lower Sixth</option>
<option>Upper Sixth Science</option>
<option>Upper Sixth Arts</option>
</select></p>
<p><input type="submit" value="Register"></p>
</form>
```

Role of the `name` attribute: it identifies each piece of data when the form is sent to the

server, so that the server knows which value answers which question. Moreover, radio buttons that share the same `name` (here `status`) form a single group, which is exactly why only one of Day student and Boarder can be selected at a time.

9.3 Solutions — I prepare for the GCE

SOLUTION **Exercise 9 — Structured question: reproducing a page (20 marks)**

1. Expected code (12 marks). Indicative mark scheme: `<!DOCTYPE html>` (1); `<html>` correctly opened and closed (1); `<head>` containing `<title>GCE Revision</title>` (2); `<body>` (1); `<h1>` heading (1); two paragraphs (2); the word free in bold with `<b>` (1); bulleted list `<ul>` with three `<li>` (2); image `<img src="study.png" alt="...">` (1).

HTML code

```
<!DOCTYPE html>
<html>
<head>
<title>GCE Revision</title>
</head>
<body>
<h1>Welcome to GCE Revision</h1>
<p>This site helps Upper Sixth students prepare for the examination.</p>
<p>All the resources on this site are <b>free</b>.</p>
<ul>
<li>Mathematics</li>
<li>Physics</li>
<li>ICT</li>
</ul>
<img src="study.png" alt="Students revising for the GCE">
</body>
</html>
```

2. Purpose of `alt` (3 marks). It provides an alternative text that is displayed when the image cannot be loaded (missing file, slow connection) and that is read aloud by screen readers for visually impaired users. A suitable value here: `alt="Students revising for the GCE"` (1 mark for each role, 1 mark for a sensible value).

3. `<title>` versus `<h1>` (3 marks). `<title>` is placed in the `<head>` and appears in the

browser tab, not in the page itself; `<h1>` is placed in the `<body>` and appears as the large visible heading inside the page. (1 mark for the location of each element, 1 mark for the contrast in display.)

4. Modified line (2 marks). `<li><a href="ict.html">ICT</a></li>` — a relative URL is correct because `ict.html` is in the same folder (1 mark for `<a href="ict.html">`, 1 mark for keeping it inside `<li>`).

SOLUTION **Exercise 10 — Error hunting (18 marks)**

1. The six mistakes (12 marks: 1 mark for identification + 1 mark for correction, each).

Line	Mistake	Correction
3	Closing tag lacks the slash: `<title>` instead of `</title>`	`<title>My page</title>`
7	The paragraph is never closed	`<p>Welcome to our site</p>`
11	`<li>` is placed outside any list	Move it inside the list, before line 10
12	The attribute value is not enclosed in quotes	`<img src="photo.png">`
12	The `alt` attribute is missing	`<img src="photo.png" alt="School photo">`
13	Closing tag lacks the slash: `<a>` instead of `</a>`	`<a href="contact.html">Contact</a>`

2. File extension (3 marks). The extension `.txt` declares the file as plain text, so the browser displays the source code as it is instead of interpreting the tags (2 marks). The file must be saved as `page.html` (or `page.htm`) (1 mark).

3. Why add `alt` (3 marks). Any two of: it is read aloud by screen readers for visually impaired visitors; it is displayed in place of the image when the file is missing or cannot be downloaded (e.g. on a slow connection); it helps search engines understand the content of the page.

SOLUTION **Exercise 11 — Essay question: the school web site (20 marks)**

Examiner's expectations and indicative mark scheme:

1. Structure of a well-formed HTML document (6 marks). Award 1 mark per element correctly named and explained: `<!DOCTYPE html>` declares the document type so the browser interprets the file as HTML; `<html>` is the root element containing the whole

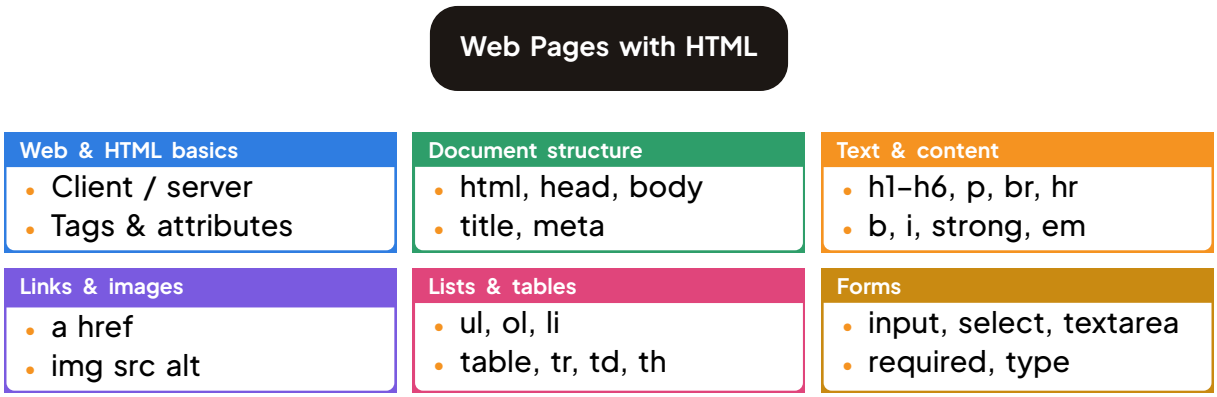
document; `<head>` holds information about the page that is not displayed; `<title>` (inside `<head>`) names the page in the browser tab; `<body>` holds the visible content (headings, paragraphs, images, links, forms). The final mark is for stating that every opened tag must be closed in the reverse order of opening.

2. Absolute versus relative URL (6 marks). An absolute URL gives the complete address: protocol, server name and full path, e.g. `https://www.minesec.gov.cm/results.html` (2 marks: definition + example); it is required when pointing to a resource on another site (1 mark). A relative URL gives the path from the current document, e.g. `pages/classes.html` (2 marks); it is preferable for links within the same site because the site keeps working if it is moved to another server or folder (1 mark).

3. Choice of form controls (4 marks). Name: `<input type="text">`, because the answer is free text (1 mark). Class: a `<select>` drop-down list, because the value must be chosen among a fixed set of classes, which prevents typing errors (2 marks). Sex: two radio buttons sharing the same `name`, because exactly one option must be selected (1 mark).

4. Good practices (4 marks: 2 per practice, any two). Provide an `alt` text for every image, so that screen readers can describe it and visitors still get the information if the image fails to load; compress images and keep pages light so that they load quickly on a slow connection; use a clear heading structure (`<h1>`, `<h2>`) so that assistive technologies can navigate the page; ensure a strong contrast between text and background colours for readers with low vision.

10 SUMMARY SHEET



Mind map of the chapter: developing simple web pages with HTML.

FICHE BILAN **L'essentiel à connaître pour l'examen**

Key definitions

- **Web page**: a document written in **HTML** (HyperText Markup Language) and displayed by a **web browser** (Chrome, Firefox).
- **Website**: a collection of linked web pages stored on a **web server** and reached through a **URL** via the **HTTP/HTTPS** protocol.
- **Tag**: a markup command between angle brackets, e.g. `<p> …</p>`; most tags come in opening/closing pairs; some are empty (`<br>`, `<img>`, `<hr>`).
- **Attribute**: extra information inside an opening tag, written `name="value"`, e.g. `<img src="flag.jpg" alt="Flag">`.
- **Hyperlink**: a clickable reference to another page, file or section, created with `<a href="...">`.
- **Form**: an interactive zone collecting user data and sending it to a server (`action`) by a method (`GET` or `POST`).

Essential structure of an HTML document

- Skeleton: `<!DOCTYPE html>`, then `<html>`, containing `<head>` (invisible information: `<title>`, `<meta charset="UTF-8">`) and `<body>` (visible content).
- Text: headings `<h1>` to `<h6>`, paragraph `<p>`, line break `<br>`, horizontal rule `<hr>`; emphasis `<b>`, `<i>`, `<strong>`, `<em>`.
- Lists: unordered `<ul>`, ordered `<ol>`, items `<li>`; definition lists `<dl>`, `<dt>`, `<dd>`.

- Tables: `<table>` with rows `<tr>`, header cells `<th>`, data cells `<td>`; spanning with `colspan` and `rowspan`.
- Links: `<a href="page2.html">`, external `href="https://..."`, anchor `href="#top"`; images: `<img src="..." alt="..." width="...">`.
- Forms: `<form action="..." method="post">` containing `<input type="text|password|radio|checkbox|submit|reset">`, `<select>` with `<option>`, `<textarea>`, and `<label>`.

Methods to master

1. Create a page: open a text editor (Notepad, Notepad++), type the code, save with extension `.html`, open in a browser to test.
2. Always write the full skeleton first, then fill the `<body>`.
3. Validate by checking: every opened tag is closed, attribute values are in quotes, file names and paths of images are exact.
4. Simple validation of forms: attributes `required`, `type="email"`, `type="number"`, `min`, `max`, `maxlength`.

Common mistakes to avoid

- Forgetting the closing tag (`</td>`, `</li>`) or the slash: `<p>` is not closed by `<p>`.
- Putting visible text in the `<head>` or forgetting `<title>`.
- Wrong image path: the image must be in the same folder (or correct relative path) as the page.
- Confusing `<ul>` (bullets) with `<ol>` (numbers), or `<th>` (bold centred header) with `<td>`.
- Omitting quotes around attribute values, or misspelling `href` as `scr`/`src` as `scr`.
- Using `method="get"` for sensitive data (passwords): values appear in the URL; prefer `post`.

GCE tips

- Be able to write by hand the complete skeleton of a page and a small form: this is a frequent practical and theory question.
- Know how to read given HTML code and predict what the browser displays (headings hierarchy, table with `colspan`, nested lists).
- Cite concrete Cameroonian uses: school result portals, e-government services, mobile money web interfaces (MTN MoMo, Orange Money) all rely on HTML forms.
- In answers, define HTML precisely (a markup language, not a programming language) and give one correct tag example per question.

Signed • OnBuch+
ONBUCH+ TEACHING TEAM
FOLLOWS THE OFFICIAL MINESEC SYLLABUS

OnBuch+